

BİLKENT UNIVERSITY

DEPARTMENT OF ELECTRICAL AND ELECTRONICS
ENGINEERING

EEE-491

Electrical and Electronics Engineering Design I

Spoken Number Recognition Project

CUMULATIVE LAB REPORT

Prepared by

Cankut Budak

Melih Kutay Yağdereli

10.01.2025

TABLE OF CONTENTS

1 PROJECT OBJECTIVE AND SPECIFICATIONS	4
LAB DEBUG	4
LAB CONTROL	4
LAB ADC	4
LAB WINDOW	5
LAB PCB	5
LAB MATLAB	5
LAB FFT	5
LAB MEL	5
LAB DCT	6
LAB COMPARE	6
2 SYSTEM DESCRIPTION	6
LAB DEBUG	6
LAB CONTROL	7
LAB ADC	7
LAB WINDOW	9
LAB PCB	9
LAB MATLAB	10
LAB FFT	10
LAB MEL	11
LAB DCT	12
LAB COMPARE	12
3 IMPLEMENTATION and TESTS	13
3.1 DEBUG	13
3.2 CONTROL	13
3.3 ANALOG DIGITAL CONVERSION	14
3.4 WINDOW	15
3.5 PCB	15
3.6 MATLAB	22
3.7 FAST FOURIER TRANSFORM	25
3.8 MEL FILTER BANKS	25
3.9 DISCRETE COSINE TRANSFORM	26
3.10 COMPARE	26
4 DISCUSSION AND CONCLUSION	26
4.1 Conclusion	26
4.2 Encountered Problems	27
4.3 Future Work	28
5 REFERENCES	28
6 APPENDICES	29

1 PROJECT OBJECTIVE AND SPECIFICATIONS

In this project, we developed a spoken number recognition system that utilizes mel cepstral coefficients alongside the L2 norm for each digit. To accomplish this, 2 second voice clips were split into 63 frames, each spanning roughly 20 ms [1]. In order to mitigate the leakage issue commonly encountered when performing the Fast Fourier Transform (FFT), the frames were overlapped by 50% and subjected to windowing. Next, mel filter banks were applied to transform the signals into a domain that better aligns with human auditory perception, particularly in discerning changes at lower frequencies [1]. These extracted features were then compared using the Euclidean distance metric, identifying the most probable result. The entire system was realized on a Basys3 FPGA platform, with corresponding laboratory experiments carried out as follows:

LAB DEBUG

This module is responsible for checking the correct implementation of sub-blocks. This lab implementation is used for providing the data transmission between our FPGA and MATLAB. It uses the UART serial communication protocol (Universal Asynchronous Receiver-Transmitter) with the baud rate of 115200 [2]. Throughout the implementation process, we used this module to ensure that our sub-blocks work as desired in the lab manuals. As each sub-block is completed, we adapted this module to establish successful data transmission.

LAB CONTROL

This module of the project is responsible for uniting the whole sub-blocks of the system. The main purpose of lab control is to successfully manage the signals coming from and going to each module. It ensures that the every ready and start signal of each module is handled. Also, lab control ensures that the %50 frame overlapping of incoming signals is completed. Throughout the implementation process, this module is updated and each sub-block is integrated. With this block the cycle of windowing, FFT and MEL is implemented correctly.

LAB ADC

This module functions as the interface between the ADC121S101 12-bit A/D converter and the system. Its primary purpose is to acquire the spoken numbers 12-bit audio signal sampled at 16 KHZ from the ADC121S101 A/D converter then storing these signals on a dual port RAM after decimating and applying a pre-emphasis filter. It ultimately collects 16384 samples corresponding to 1.5 seconds of speech signal and writes it to the dual port RAM. Communication with the ADC board is done by the Serial Peripheral Interface (SPI) ensuring that specific chip select timings are met for accurate data collection. The incoming data is captured as 12-bit signed values using two's complement notation, for matching input specifications of the FFT block. The collected samples are then stored for further processing in the system.

LAB WINDOW

This module takes the 12 bit data from ADC modules dual port RAM as its input then applies a Hanning window to the frames of 512 samples with a %50 overlap between consecutive frames. After reading and multiplying each data with Hanning window coefficients the module writes the resulting data to a RAM as output. The overlapping frames ensure that there is a smooth transition between each frame.

LAB PCB

This module's purpose is to design a circuit which will amplify the spoken number signal with an anti-aliasing filter to ensure that the spoken signal is in the frequency range of 100 Hz to 4 KHz. To do this, 3 stages of operational amplifiers are used. The anti-aliasing filter exhibits an 80 dB/decade attenuation and maintains a gain value between -1 to 1 dB from 100 Hz to 3KHz. The 3 dB cut-off frequency of the filter is desired to fall between 4 to 4.5 KHz. The aim of this lab was to amplify the voice signal to make it oscillate between 0V and 3.3V to implement a successful analog to digital conversion in the ADC sub-block. A PCB board is designed with the Altium Designer software [3].

LAB MATLAB

This lab provides a comprehensive simulation for the entire system. Sub components specifications integrated to the MATLAB environment to gain better understanding of the project and generate required COE files and develop methods for improving the systems performance mainly focused on Mel-Frequency Cepstrum approach. All tasks for this module are carried out in MATLAB.

LAB FFT

This module is used for calculating the short-time Fourier Transform (STFT). The recorded spoken number signals are divided into six frames and 512 point fast fourier transform (FFT) is computed for each frame using Vivado's built-in IP catalogue. To ensure the windowing is done correctly, size 256 of random-access memory (RAM) is used for storing the computed results again using the Vivado's IP catalogue.

LAB MEL

This module takes its input from FFT modules RAM and applies 13 MEL filter banks. The module computes the band energies of the wave form by multiplying MEL coefficients with the FFT coefficients. This operation is done to compute the Mel-Frequency Cepstral Coefficients (MFCC). These MFCC values are stored in 32-word RAM as output of the system.

LAB DCT

This module takes the band energy data calculated in the MEL module as its input then applies Discrete Cosine Transform to these values. This operation further compresses the key features needed for digit recognition. This module finalizes the feature extraction process of the system and provides the final feature values. The results are stored in RAM as output of the system.

LAB COMPARE

This module's purpose is to record voice samples of spoken digits from 0 to 9, corresponding to a 4-bit input signal. These input signals are compared with the already recorded and stored data of spoken numbers. The existing numbers are compared with the user input signals and the spoken number is decided with respect to selecting the minimum euclidean distance on the Basys3 FPGA.

2 SYSTEM DESCRIPTION

LAB DEBUG

Header for Start data				First 32-bit data transmission				Second 32-bit data transmission					Header for Stop data			
55	AA	CC	03	1st 8-bit of First 32-bit data	2nd 8-bit of First 32-bit data	3rd 8-bit of First 32-bit data	4th 8-bit of First 32-bit data	1st 8-bit of Second 32-bit data	2nd 8-bit of Second 32-bit data	3rd 8-bit of Second 32-bit data	4th 8-bit of Second 32-bit data	...	AA	55	03	CC

Fig.1: LAB-DEBUG Protocol

Lab debug is used for the verification of multiple sub-modules of the system. It was the first lab and we have implemented a debugger which provides data transmission between the FPGA and MATLAB. The transmission is implemented with the UART protocol and connected with an USB cable. In the implementation process, UART is used for transmitting the 32-bit data. For this transmission the defined start and stop messages are instructed in MATLAB to understand the communication starts or ends. While transmitting, a finite state machine is implemented to divide the 32-bit data to 8-bit blocks then with the addition of start and stop bits the sent message is 10-bits long with the baud rate of 115200. Which corresponds to 867 clock cycles of basys3 FPGA. At the end of this implementation, we were able to successfully provide communication between the PC and the FPGA using the lab debug module. We have successfully transmitted the given data in the lab manual and make the lab adjustable for future adaptations.

LAB CONTROL

Aim of this lab was to implement a module which will have control over all the other sub-modules. It was the second lab we have implemented hence, it only included control signals for lab debug accurately. This module acts like it is the top module of the whole project and it is updated as we complete other labs. Other than being the top module of the whole project, this lab also does 50% frame overlapping to overcome the leakage problems later in the lab FFT. The block diagram of this module can be seen in the below figure.

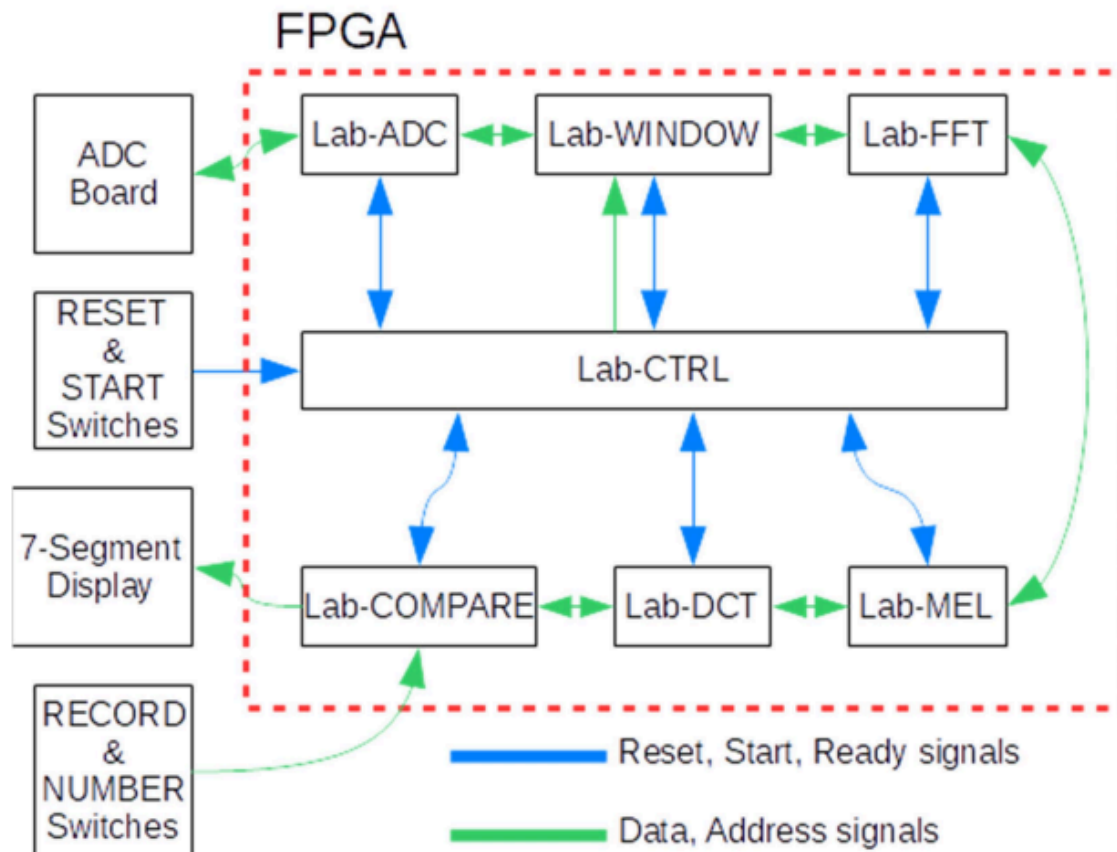


Fig.2: Lab CONTROL block diagram

LAB ADC

This module acts as an interface between the 12-bit ADC121S101 A/D converter module and the rest of the system developed in the FPGA board [4]. The communication is provided with Serial Peripheral Interface (SPI) and a clock division strategy. FPGA's 100 MHz clock divided to 12.5 MHz clock that matches with the ADC boards SPI clock. Two finite state machines (FSM) are used to manage timing for both ADC boards clock and the Chip Select (CS) signal. When CS goes low 15 bits of data read over 15 clock cycles. The first 3 bits are zero and the remaining 12 bits are the speech data acquired from the ADC board. The CS signal goes high for 4 clock signals before starting the next data frame. This operation continues until 16384

bits of data are processed. 16384 bits corresponds to 1.5 seconds of audio recording.. Pre-emphasis filter is applied by simple subtraction operation with the help of signed two's complement representation. Since ADC board samples at 500 kS/s and only 15,625 samples per second is sufficient. 32-fold decimation. These samples are stored in a 12 bit wide 16384 words deep dual port RAM for further processing in the next modules.

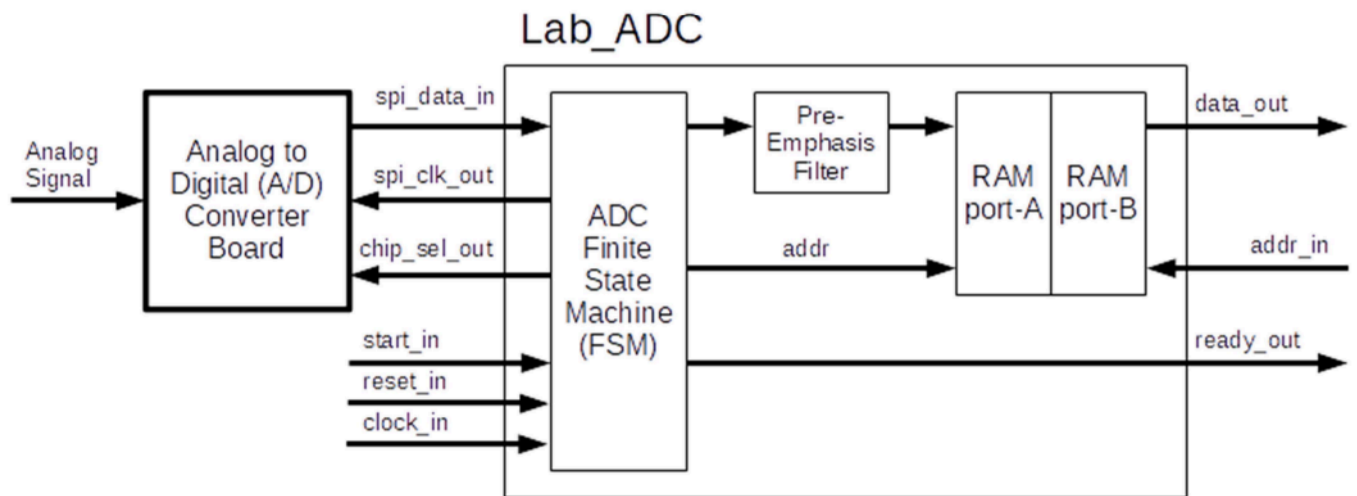


Fig.3: Lab ADC Block Diagram

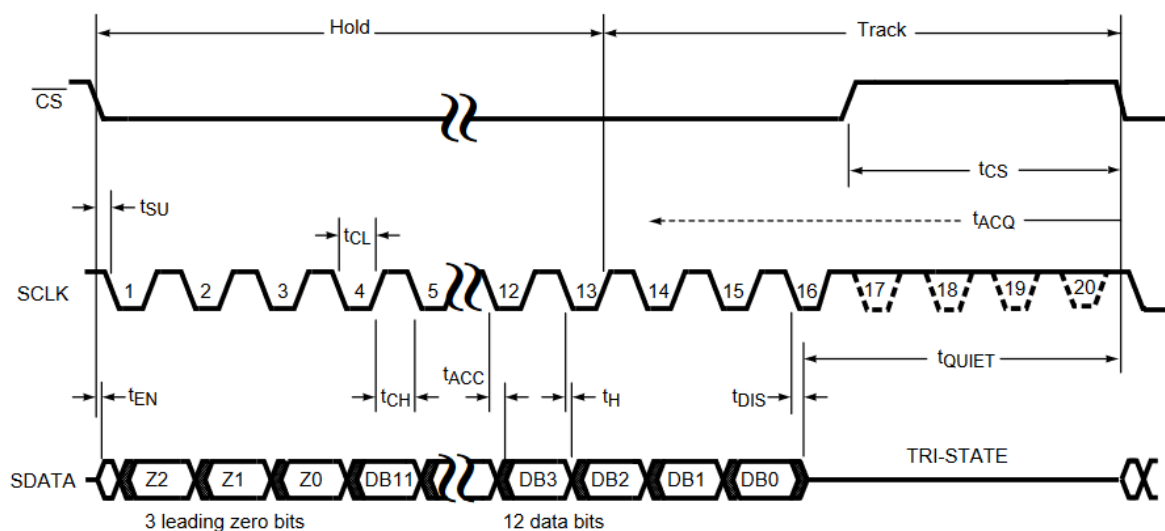


Fig.4: ADC Timing Diagram

LAB WINDOW

The lab window module processes the 12-bit output data of the ADC module. Window module organizes the data as 512 sample frames with %50 overlap. This means that each frame starts with 256 samples after the previous frame. This overlap is ensured by the control module. Within each frame the samples are multiplied with the Hanning window coefficients. These coefficients are read from 512 word 8 bit deep RAM. Then the multiplier block used for the multiplication operation. Multiplier IP block processes 12 bit data from the ADC module and the Hanning window coefficients and produces 20 bit windowed output. This operation highly reduces the spectral leakages and improves the performance for the digit recognition by smoothing the signal at the edges of each frame. At the end 16384 samples are divided to 63 frames as output of the module.

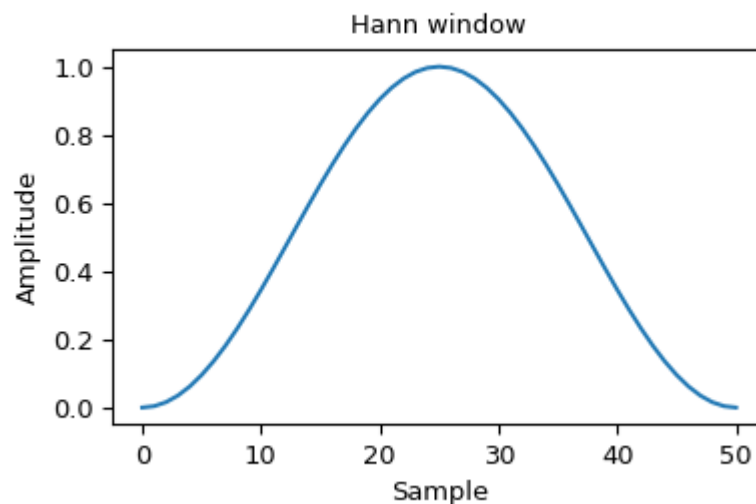


Fig.5: Hanning Window Coefficients in Time Domain

LAB PCB

Purpose of lab PCB was to design a hardware circuitry for amplifying the spoken number signal with its noise then filtering the high frequency part to obtain a more clear output that swings between 3.3V and 0V for lab ADC to read it properly. First the input signal is amplified by a LM324 Opamp then the Sallen-Key filter is applied to the signal to match the lab's requirements of 80 dB/decade attenuation with -1 to 1 dB gain between the 100 Hz and 3KHz band. The 3 dB cut-off frequency of the applied low pass filter shall be in between 4 to 4.5 KHz band. In our implementation of this lab, we first designed a circuit using LTSpice building a 3 stage opamp circuit with the first stage being the amplifier, the others are the Sallen-Key filter. After verification of this design, a PCB is drawn with the Altium Designer software. After printing the PCB, we applied different types of signals through our phone to test the circuit and observed a successful result. The circuit successfully amplified the input and checked all the requirements given. Only difference is that it

swings a little higher than the 0V level. The output of this circuit is directly connected to the ADC module we have given in order to complete the lab ADC sub-block.

LAB MATLAB

Lab MATLAB is used for simulating the end product of the FPGA based digit recognition system on the MATLAB environment. The system is based on the Mel-Frequency Cepstrum method. Matlab procedure key steps move forward as recording and storing the audio signal, segmenting the audio into frames with %50 overlap, applying Hanning window coefficients, applying FFT on each frame, multiplying the results of the FFT with Mel filter banks and storing these results for each number as a score. Then in action the system computes this score for an unknown number and compares euclidean distance between each numbers score and the unknown numbers score. The unknown number predicted as the number with the lowest euclidean distance.

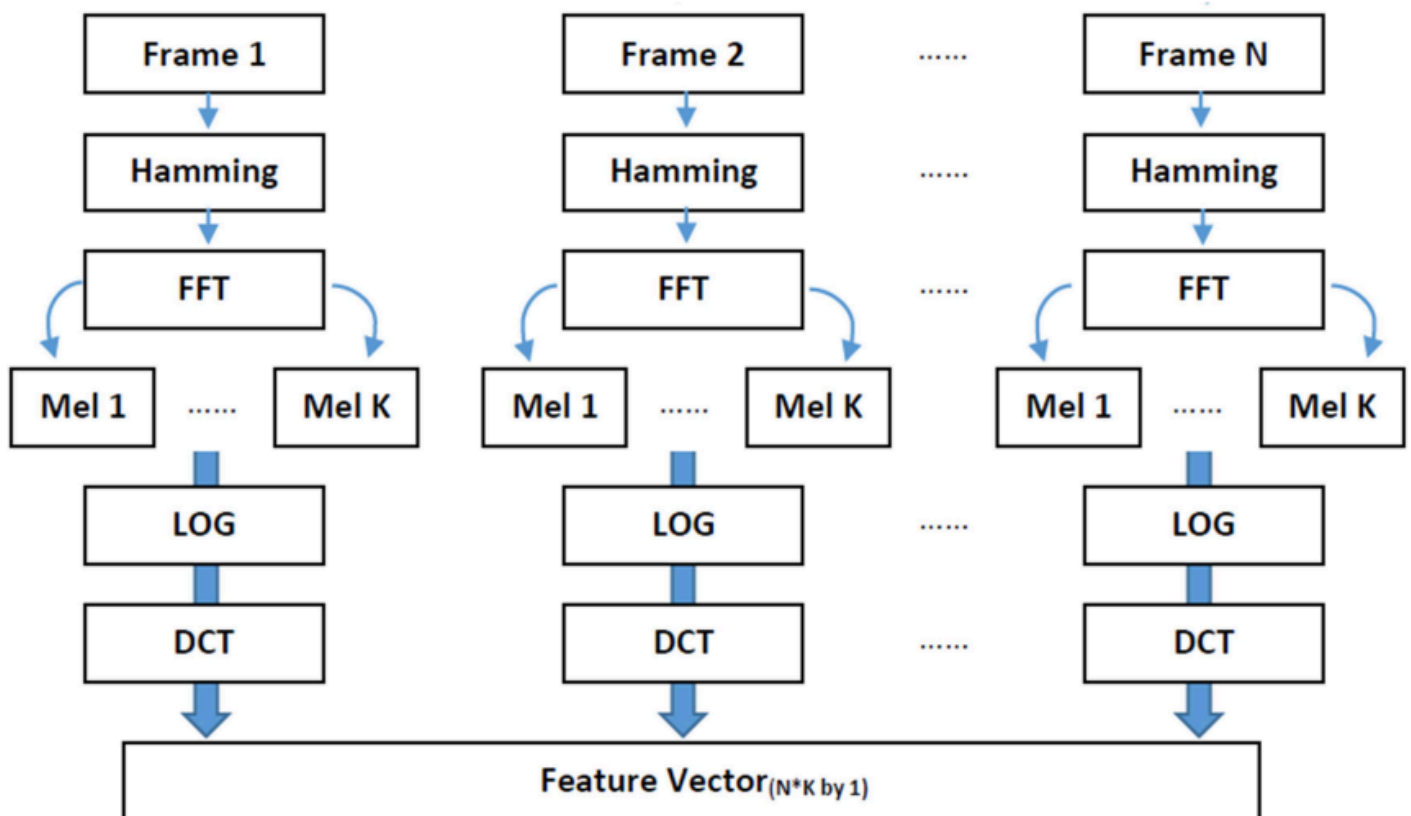


Fig.6: MATLAB diagram

LAB FFT

In this project, the FFT part of the system processes signals taken from the Lab Window section of the assignment. The incoming signal is divided into 63 frames, with each frame overlapping by 50%. The main task here is to perform the Fast Fourier Transform (FFT) on the signal. A 512-point FFT is applied, which

takes 512 input points and produces 512 output points, consisting of both real and imaginary components. Since the signal is real-valued (voice signal), only the first 256 points of the output are used, as this is sufficient due to the symmetry properties of FFT.

Instead of calculating the square root to find the magnitude of the imaginary components, we worked with the squared magnitudes. This decision was made because calculating square roots in VHDL would have been too resource-intensive, and using magnitude squares did not cause any issues in the later steps. After calculating these squared magnitudes, we stored the first 256 values in a dual-port RAM that we designed specifically for FFT data. Once this step was completed, the process continued with the Lab Mel section.

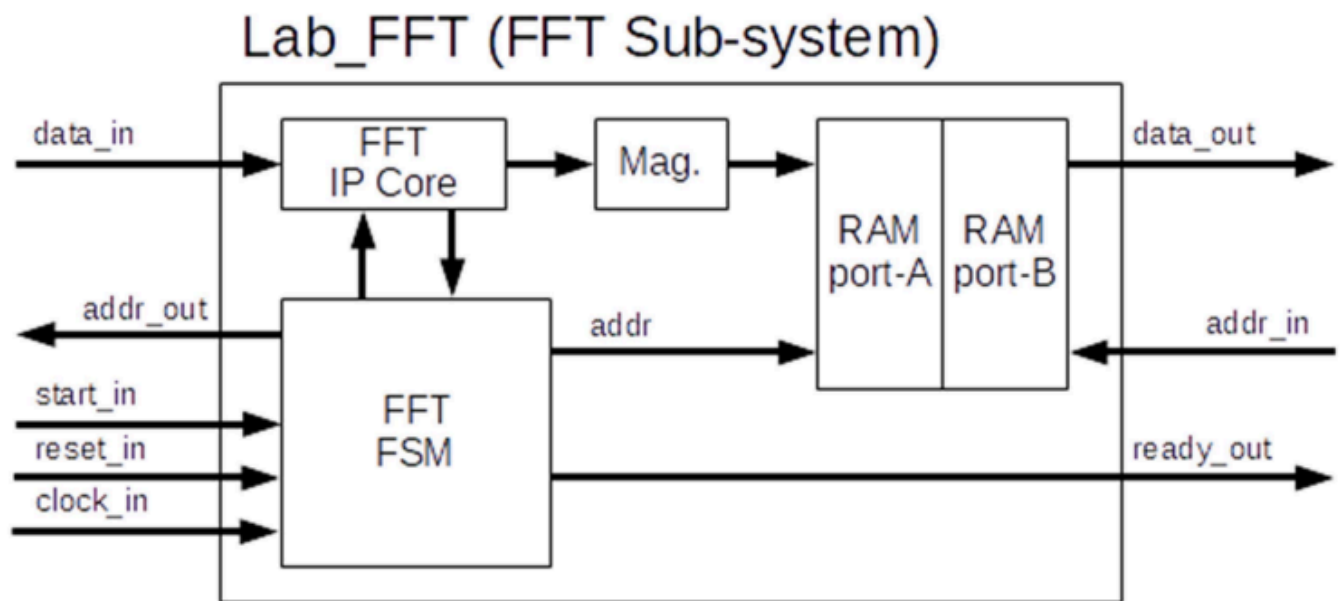


Fig.7: FFT block diagram

LAB MEL

In this module, output of the lab FFT will be applied with 32 mel filters by using the multiplier block of the Vivado. The result of this multiplication will give the band of energy waveforms. The mel coefficient is generated using the MATLAB. Purpose of this lab is to expand through the human's hearing capabilities and speaking capabilities. Filters covering through 1Hz to 4KHz which are considered as the talking frequencies will be covered. Since the FFT output is the length 512 and we are using 32 filters, that results in a total of 8192 words which will be stored in a dual-port RAM of depth 32.

LAB DCT

In this module the goal is to take the output of the lab mel which are the energies and compute their discrete cosine transform to finalize the feature extraction for the lab compare. The results shall be stored in dual-port RAM to make it achievable for the lab compare implementation.

LAB COMPARE

Lab compare can be seen as the hardware version of the lab matlab where we combined all the sub-blocks and was able to simulate spoken number recognition algorithm. In this lab all the sub-blocks are combined together and the whole system is worked simultaneously. The goal of this lab was to record the spoken digits from 0 to 9 and store their information in different RAMs built in the FPGA by the IP catalogue of Vivado. Then, test voice is recorded and stored in another RAM in order to measure the euclidean distances between the recorded information and the test sample. Minimum distance between the stored information presents the spoken number. Unfortunately we were not able to complete this lab due to earlier errors. The design would be working using the switches of the FPGA assigned for recording and testing.

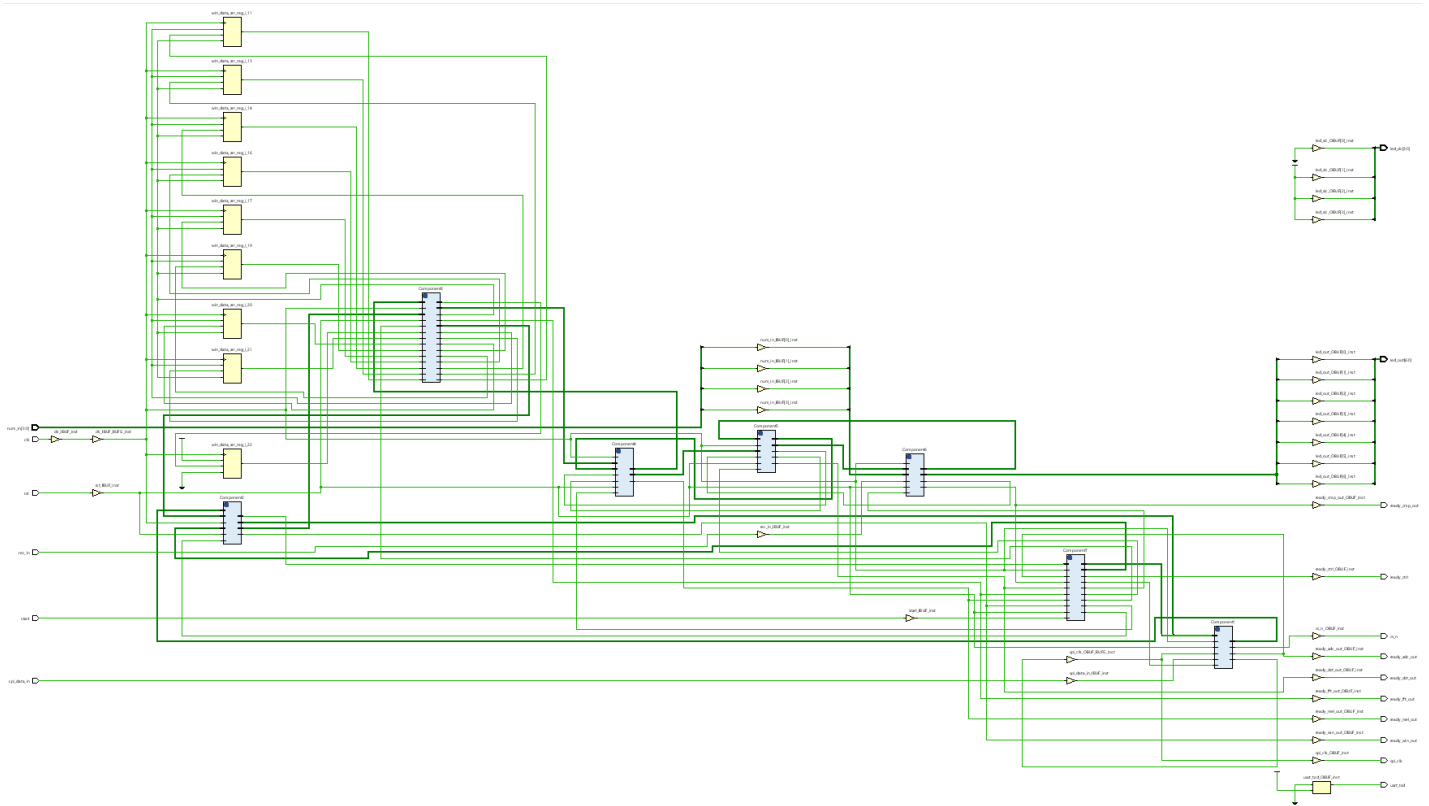


Fig.8: Desired project diagram

3 IMPLEMENTATION AND TESTS

After explaining all the sub-blocks with their essential purposes, in this section we will talk about our implementations and results. Due to problems we had from several lab assignments, unfortunately we were not able to present a hardware implementation for the spoken number recognition project. However, we have managed to implement the spoken number recognition algorithm on MATLAB. We have presented our MATLAB results and achieved a very high success rate for recognizing the spoken digits.

3.1 DEBUG

This lab is completed successfully with respect to given lab manual. UART data transmission between FPGA and MATLAB is accomplished. The simulation result can be seen in the below figure:

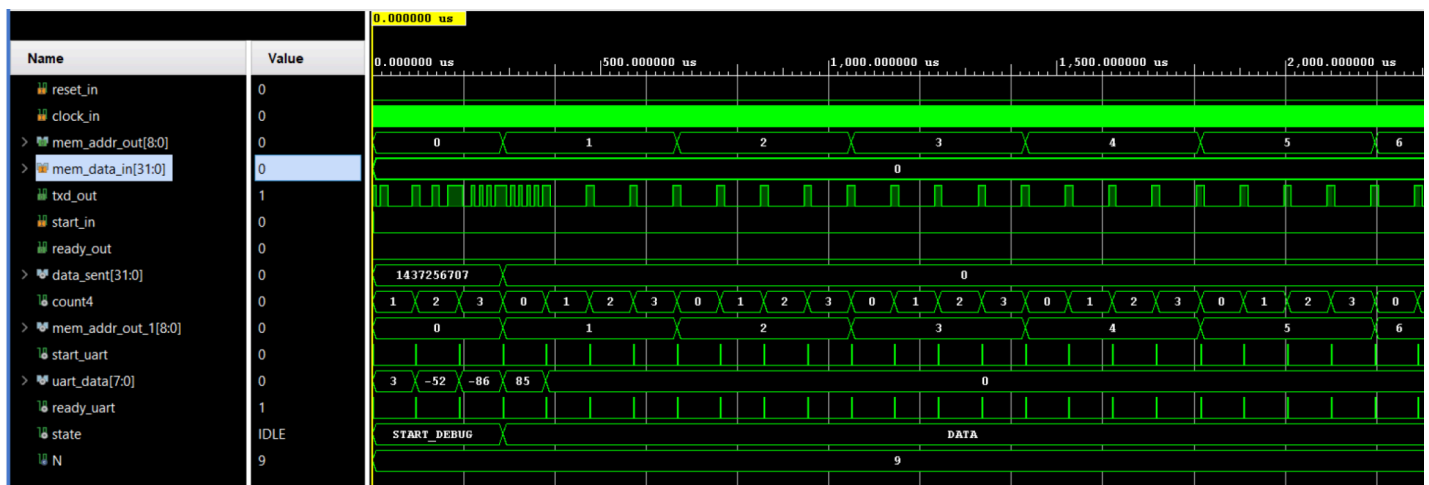


Fig.9: DEBUG simulation results

3.2 CONTROL

As described in the section 2 of this report, this works as a top module for all the sub-blocks and determines the start and ready signals conditions. Also, it successfully implements the required 50% overlapping in each frame to address the leakage problem caused by the fast fourier transform nature.

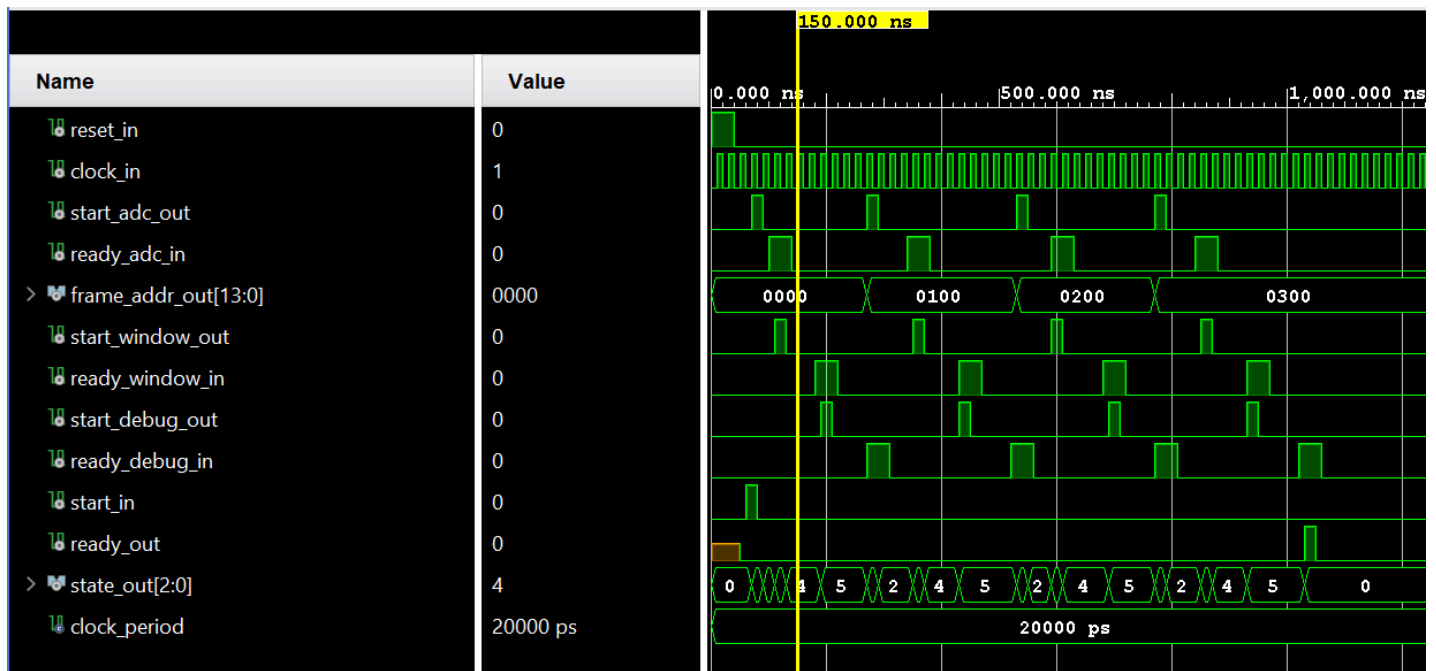


Fig.10: Lab CONTROL results

3.3 ANALOG DIGITAL CONVERSION

The implementation is done as described in section 2.3. However we never reached a fully successful implementation in this module. We successfully created a SPI clock that runs at specified 12.5 MHz from the 100 Mhz internal clock. But we were not successful in implementing a FSM that captures 1.5 second long and decimates and applies a pre pre-emphasis filter and then stores it to the dual port RAM. Our main issue was setting up the correct timing for successfully recording 15 bit frames and decimating this data. Our FSM was unable to record desired 12-bit deep 16384 words in the dual Port RAM. The results of the lab ADC can be

seen in the below figure.

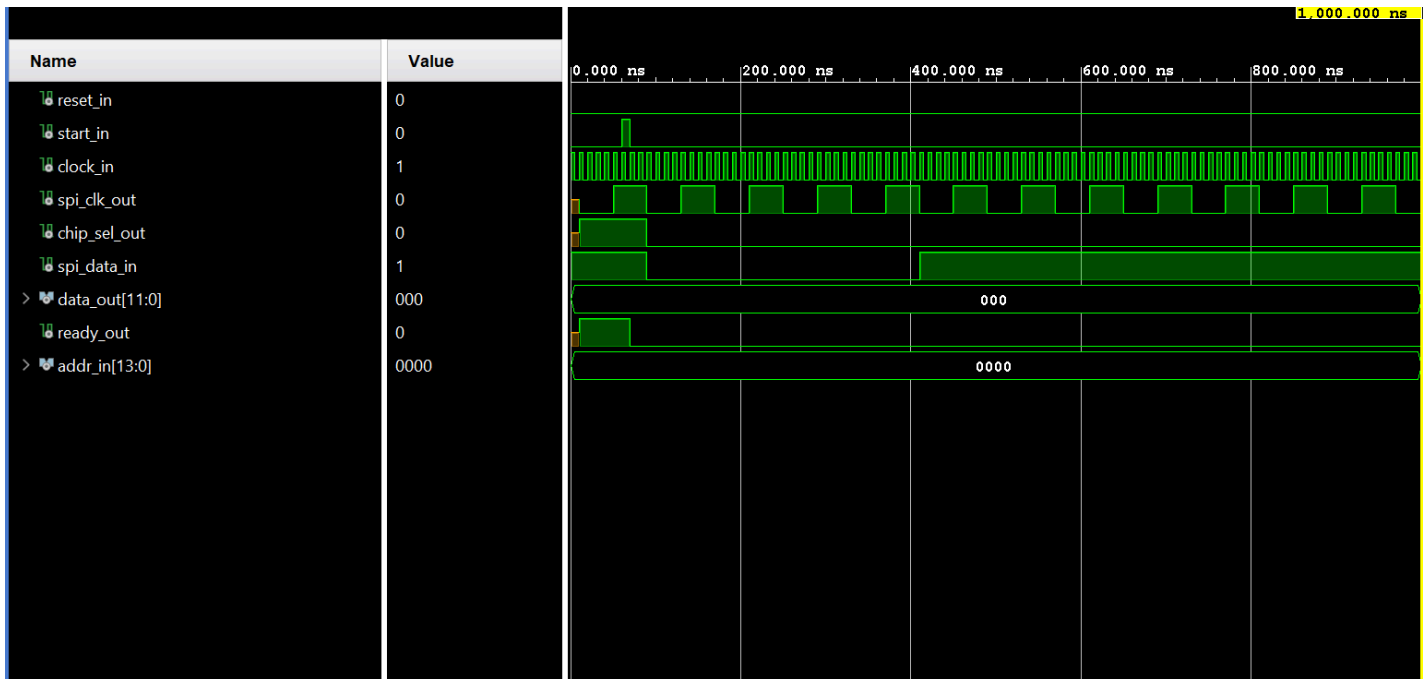


Fig.11: ADC result

3.4 WINDOW

The window lab is implemented successfully by multiplying each frame with the generated coe file from MATLAB which includes the Hanning coefficients. A dual port RAM is used for the storage of the coefficients and the multiplication results. Also, multiplier built-in block of Vivado’s IP catalogue is used for the implementation. Results can be seen in the figure below:

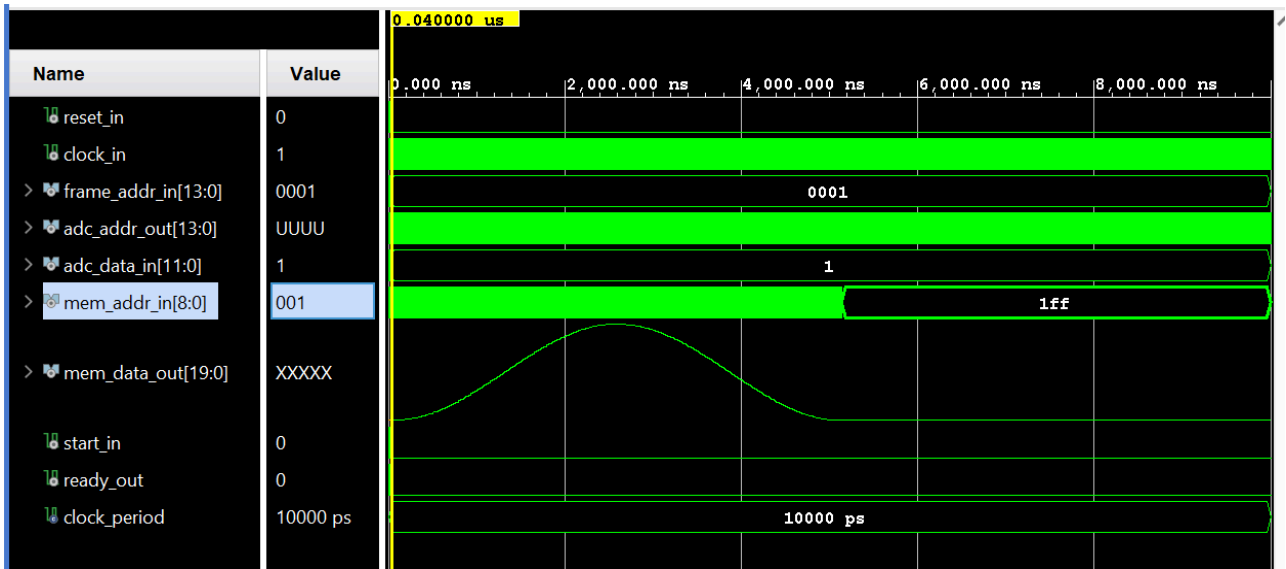


Fig.12: Window results

3.5 PCB

The amplifier and the Sallen-Key filter is accomplished successfully by first completing the software simulation then the hardware setup of the circuit is verified and the final PCB is drawn using the Altium Designer PCB design tool. The implemented circuit satisfies the given requirements successfully.

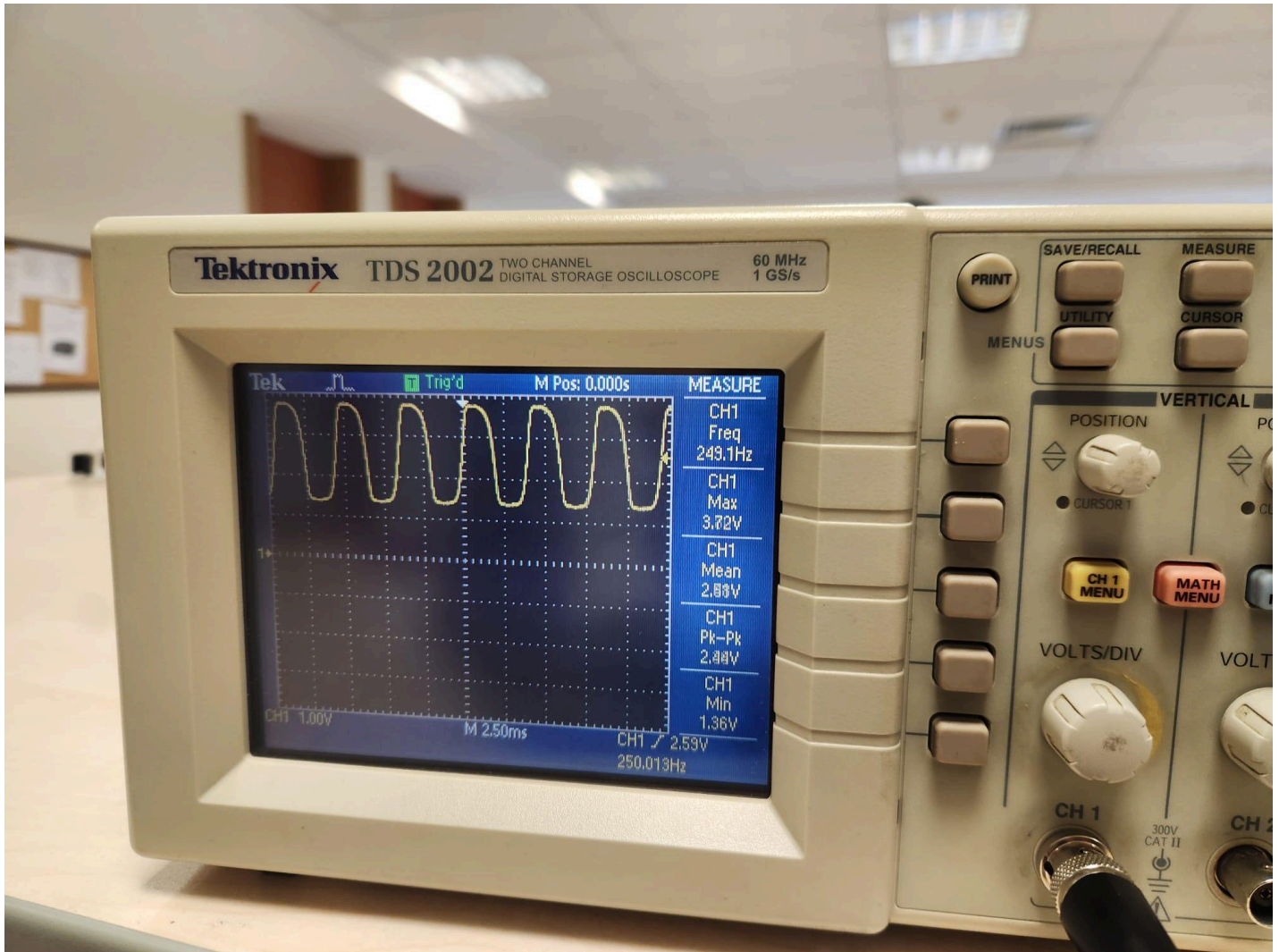


Fig.13: Output of the PCB with 250Hz signal

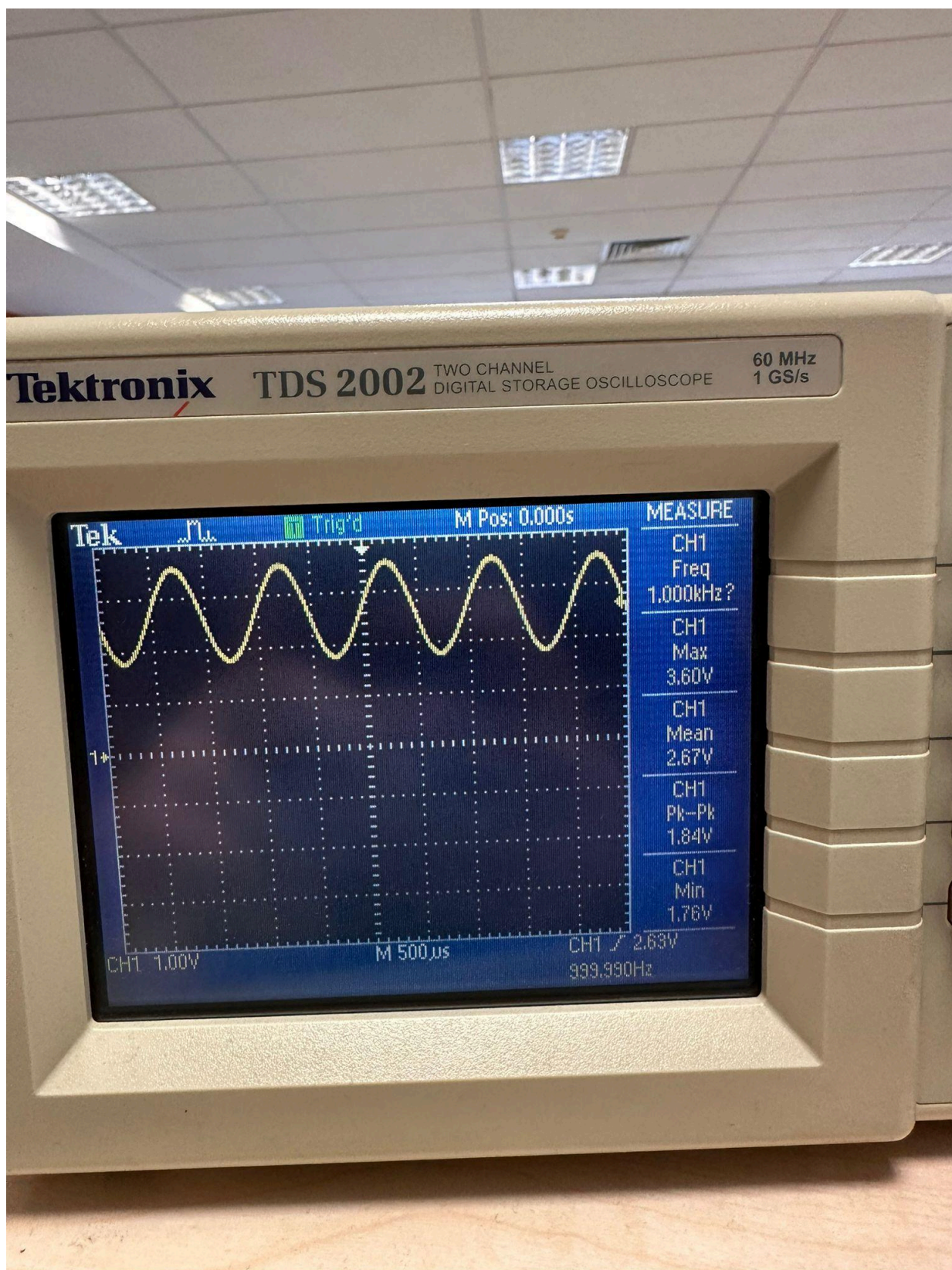


Fig.14: Output of the PCB with 1KHz signal

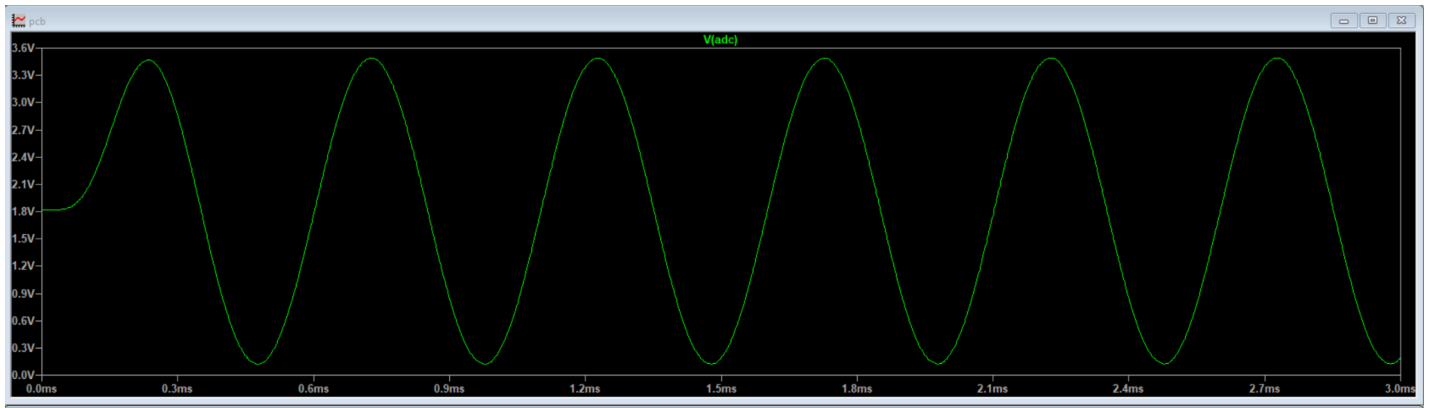


Fig.15: simulation output of the circuit

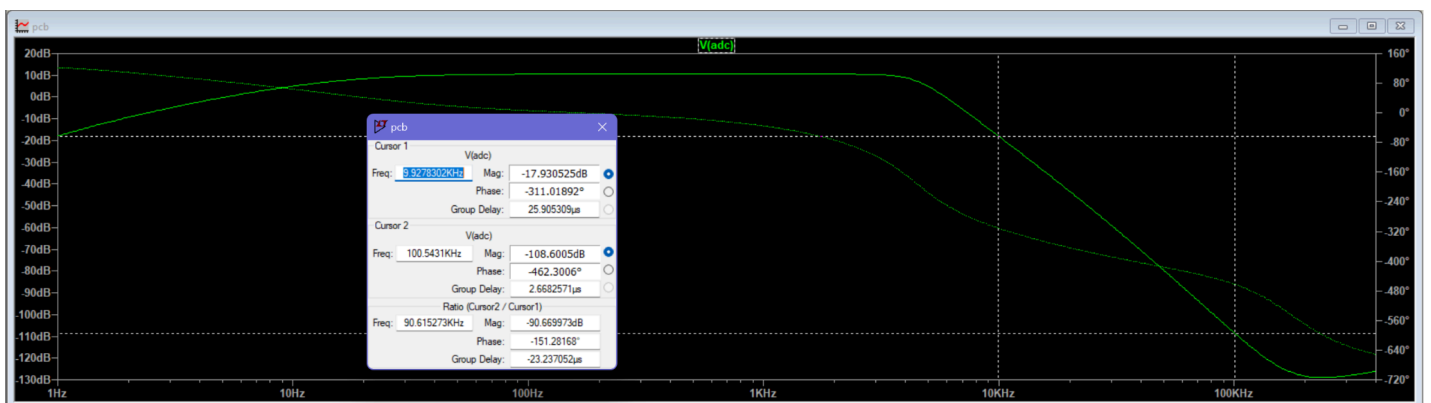


Fig.16: Frequency response of the circuit

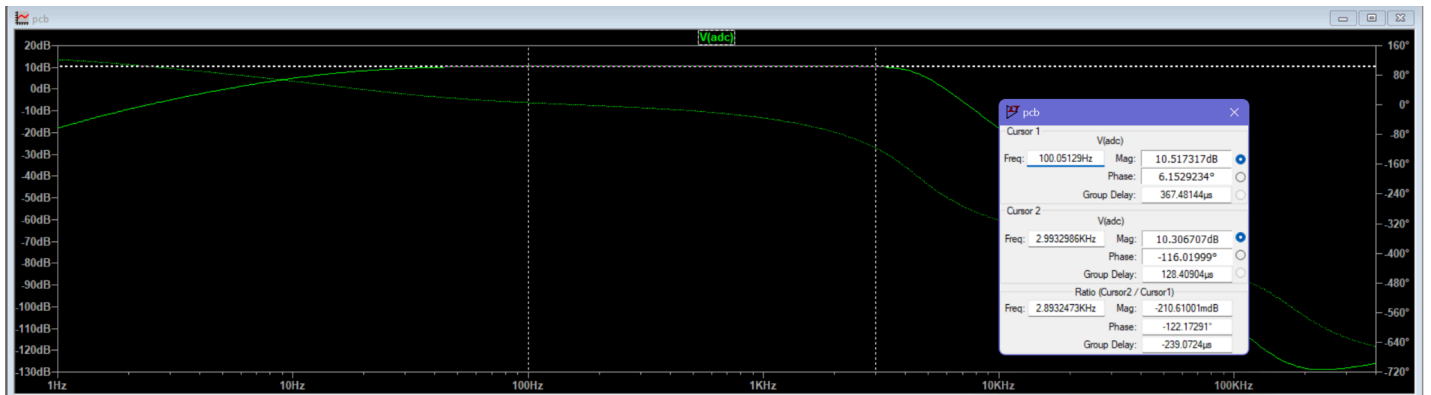


Fig.17: -1 to 1 dB gain from 100 Hz to 3 KHz

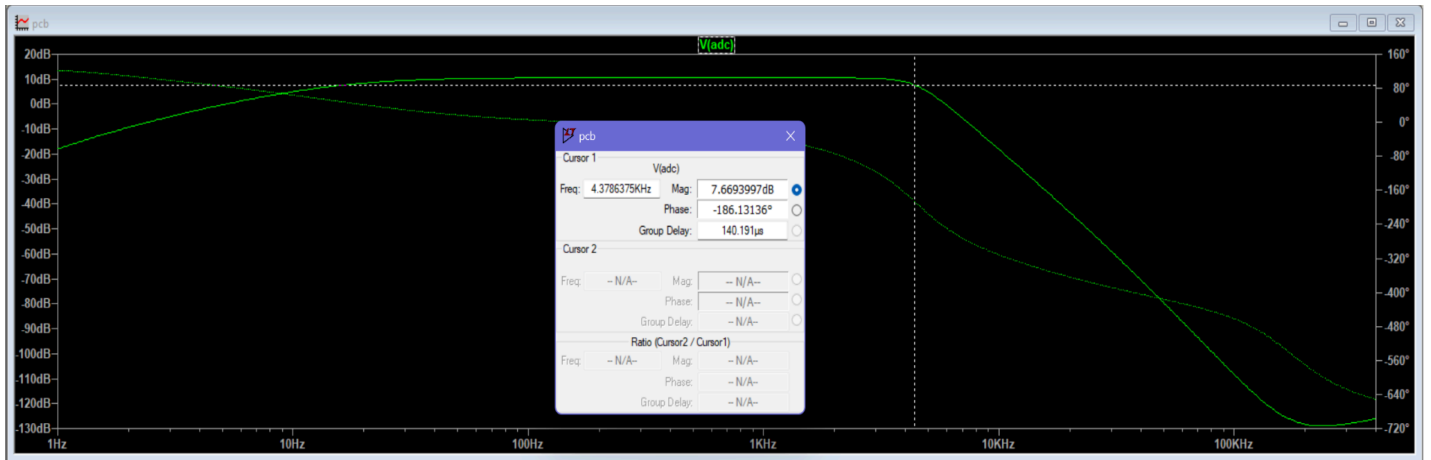


Fig.18: cut-off frequency between 4 to 4.5 KHz

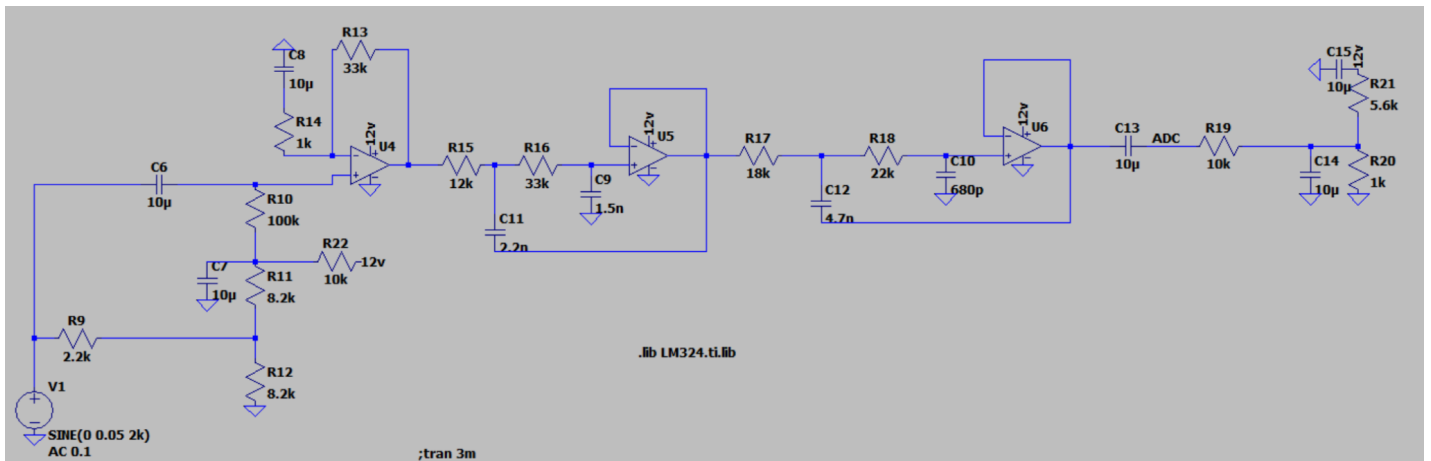


Fig.19: implemented circuit

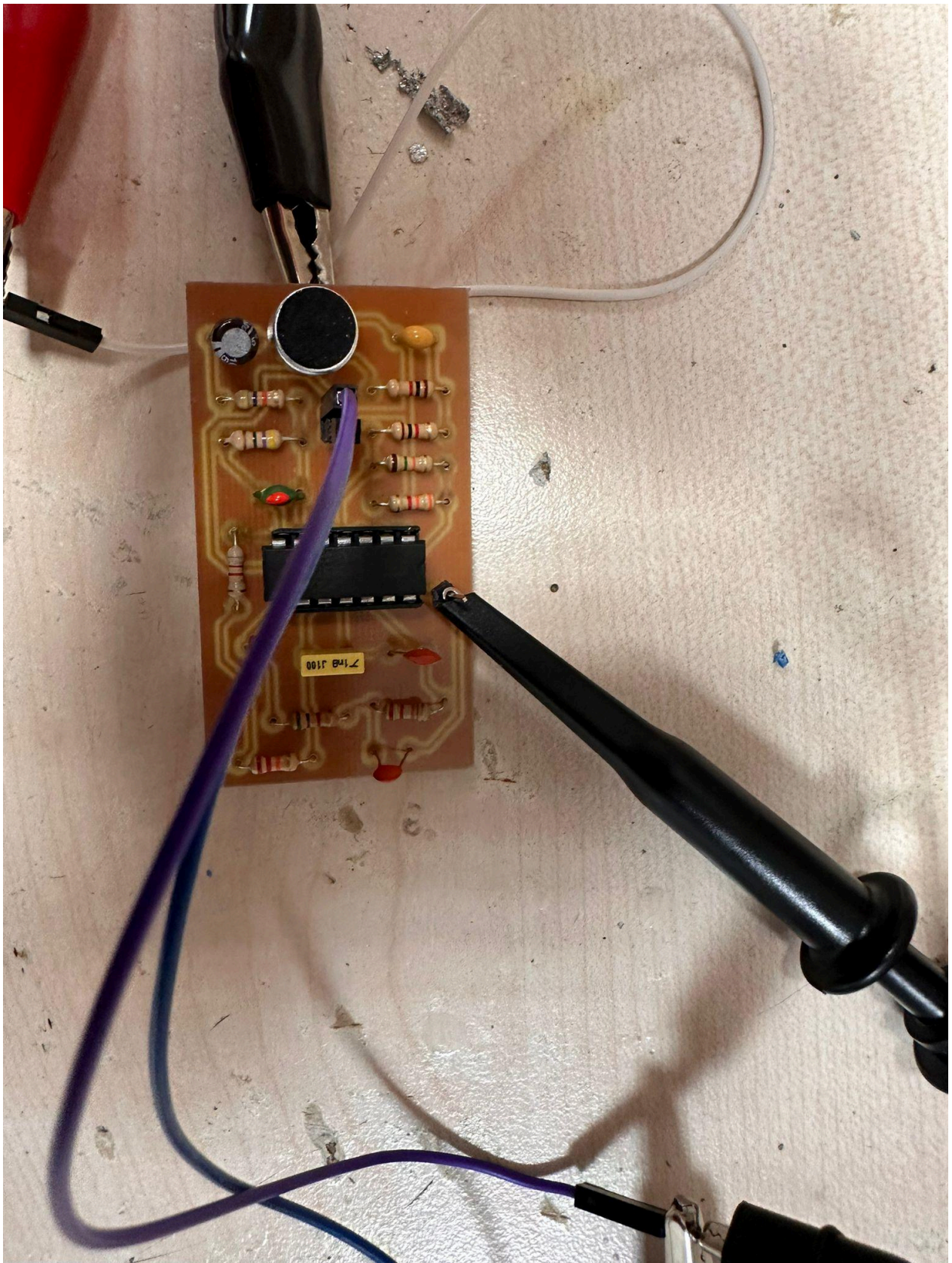


Fig.20: PCB

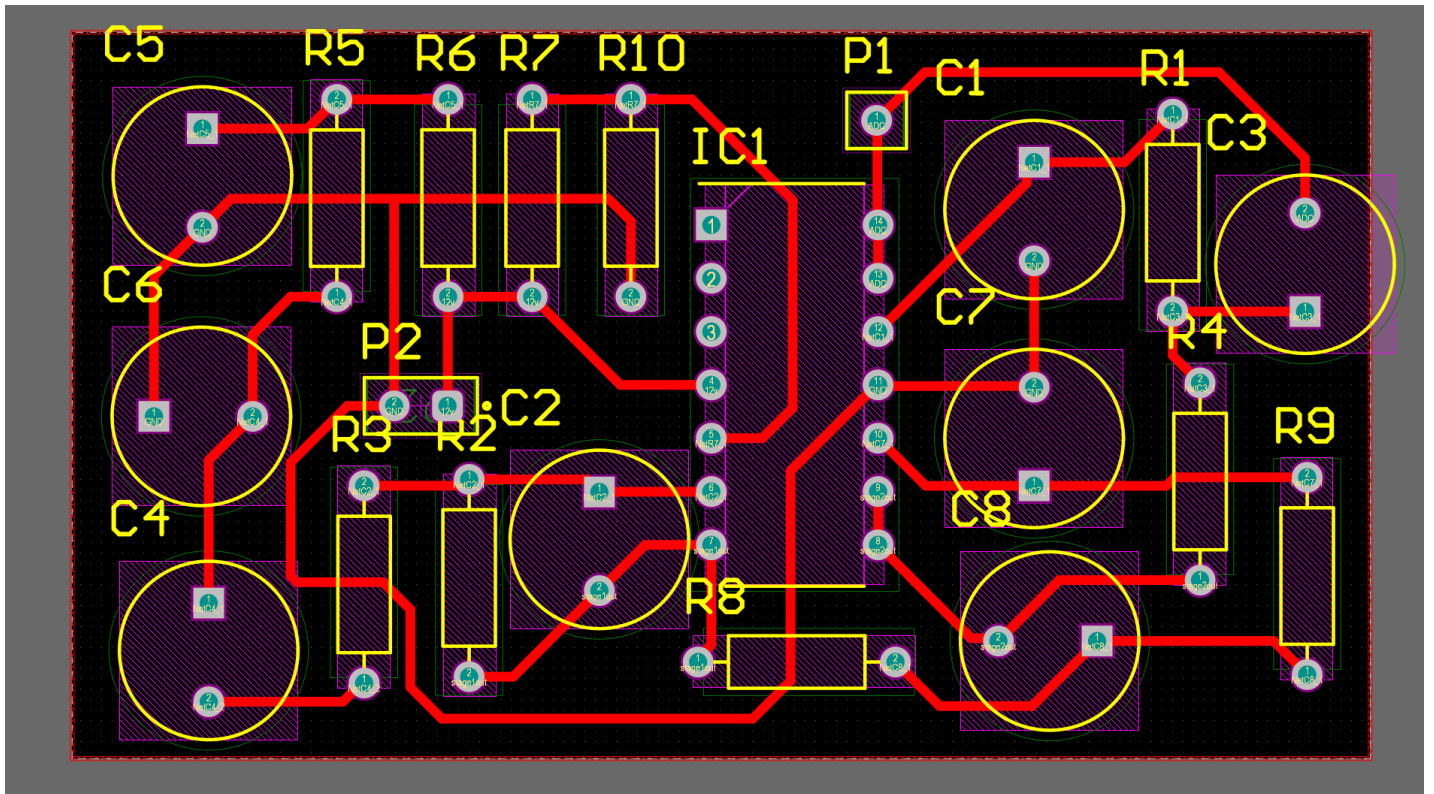


Fig.21: 2D PCB layout

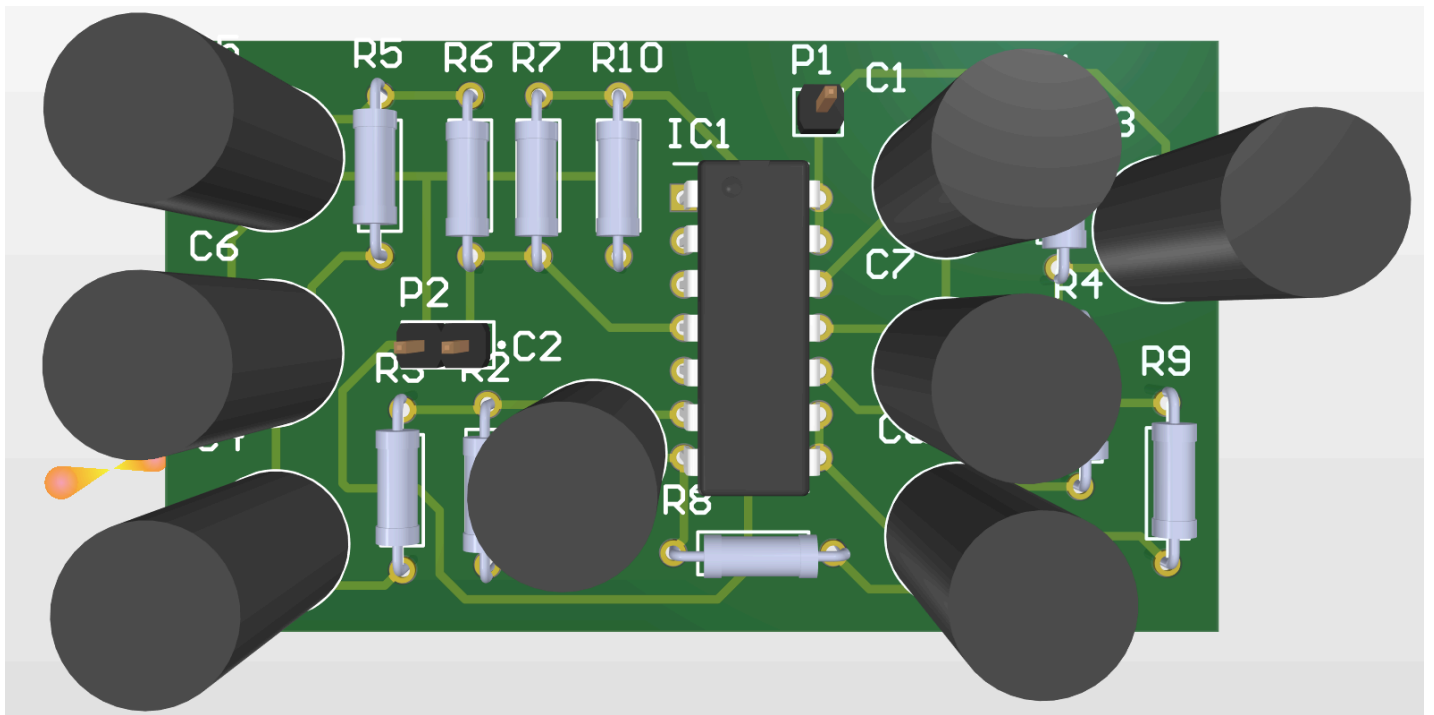
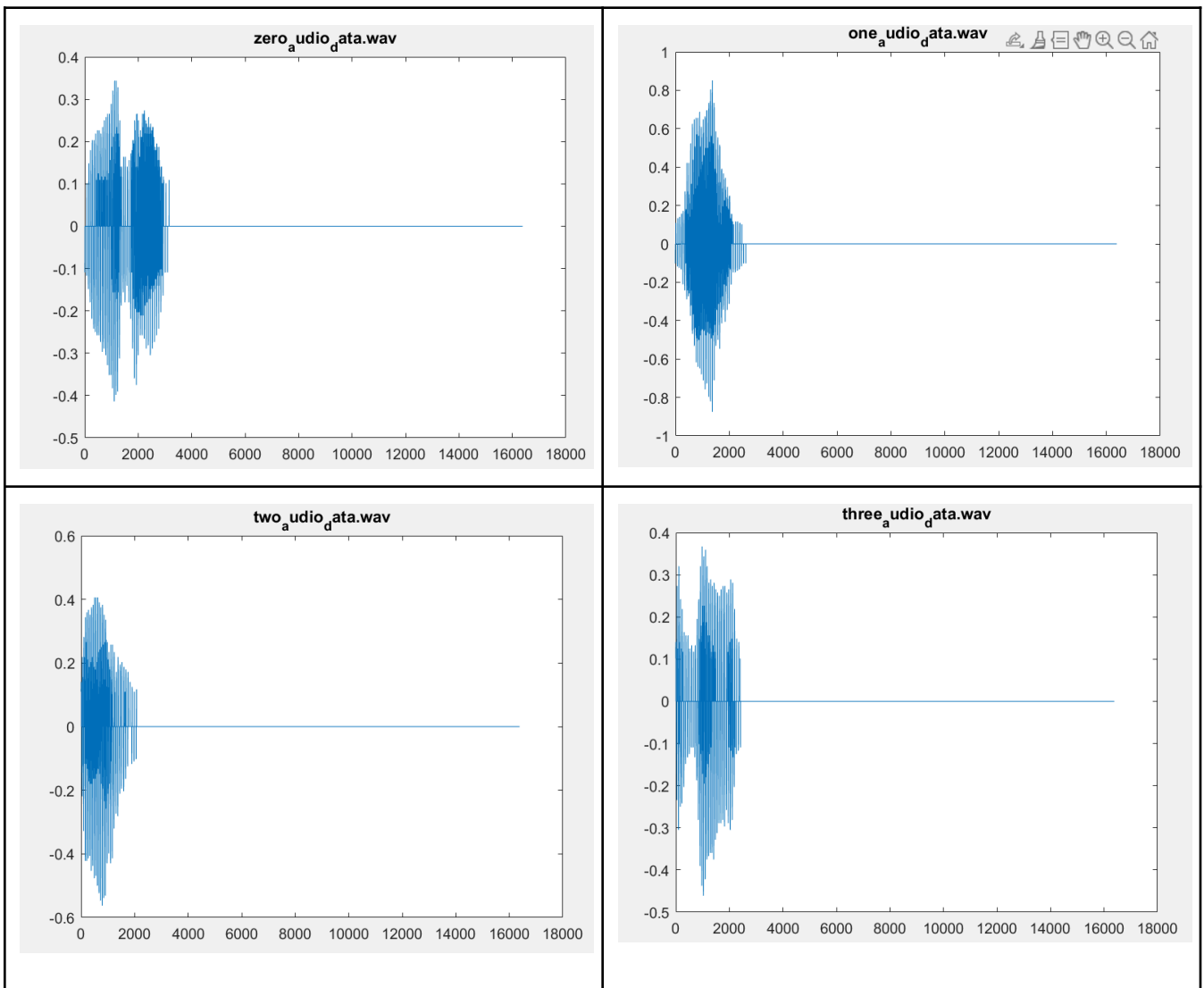
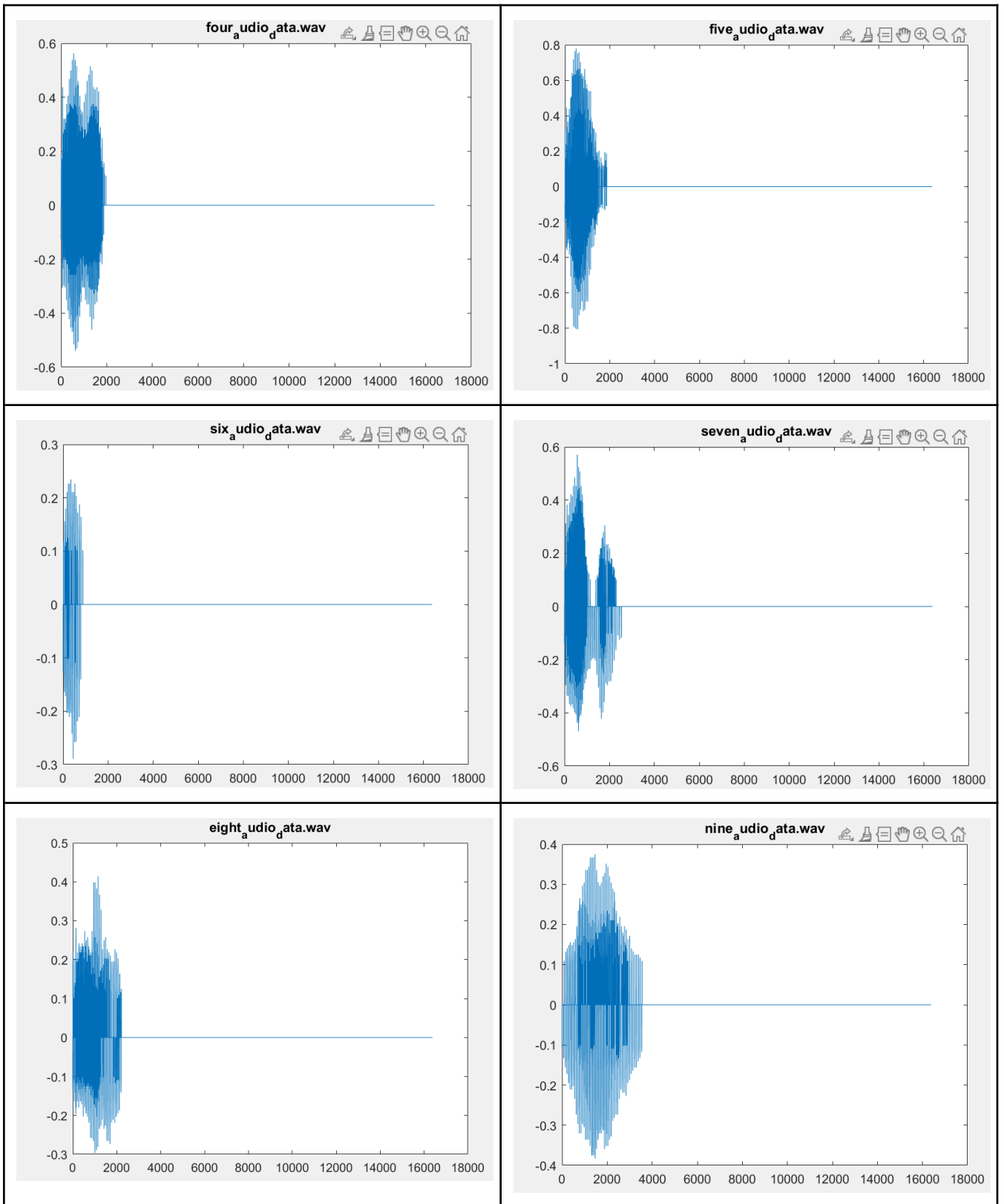


Fig.22: 3D PCB layout

3.6 MATLAB

In the implementation the procedures explained in section 2.5 are used. In addition to section 2.5 Short time Fourier transform used for better results. It replaces the %50 overlapping frame production and FFT operation. Since STFT inherently frames the data it is easier to write and it increases our accuracy predicting the spoken digit [6]. Our module at the end has reached 29/30 accuracy which is an almost perfect score.





Figures 23 to 32 displays the recorded audios.

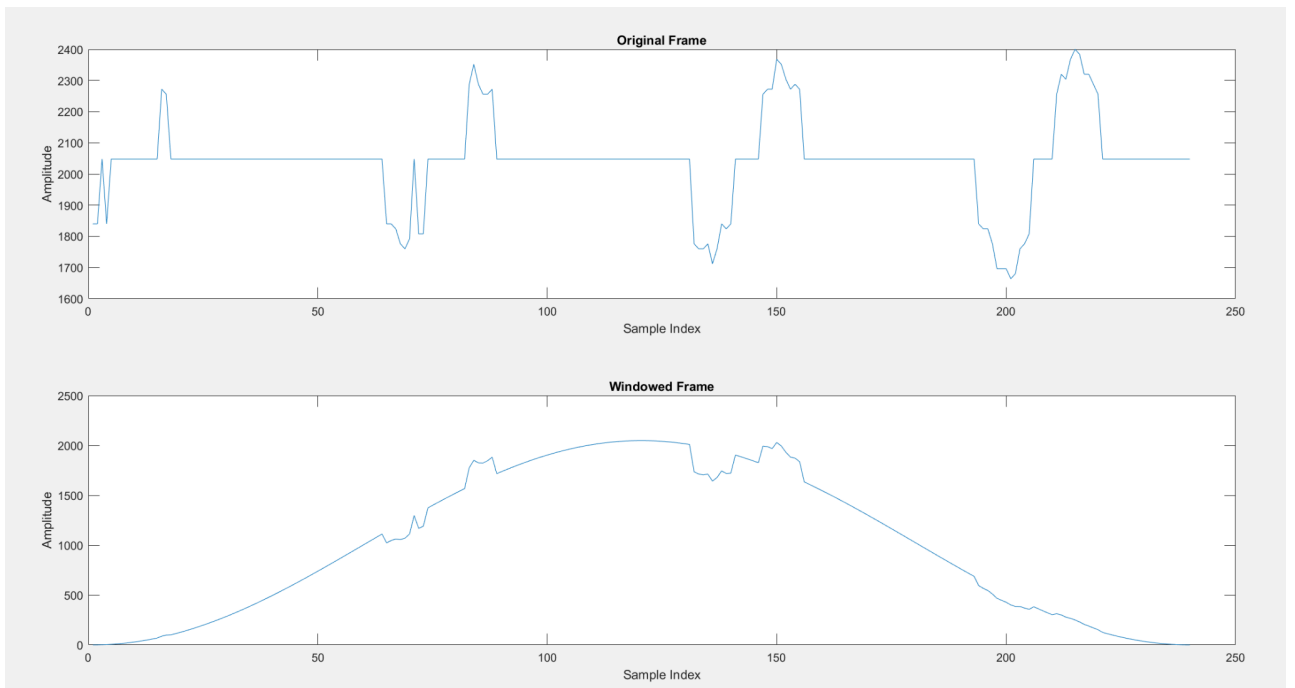


Fig.33: Windowing in MATLAB

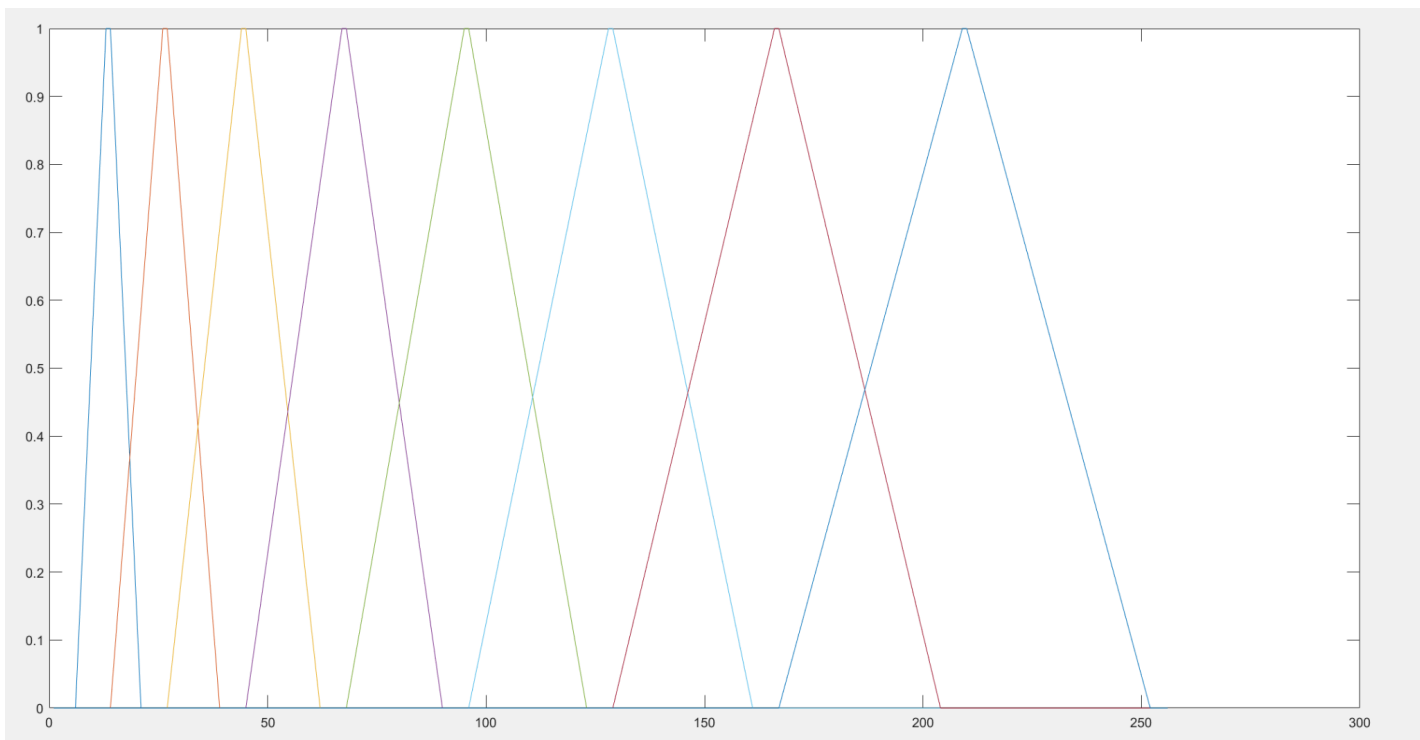


Fig.34: MEL filter banks

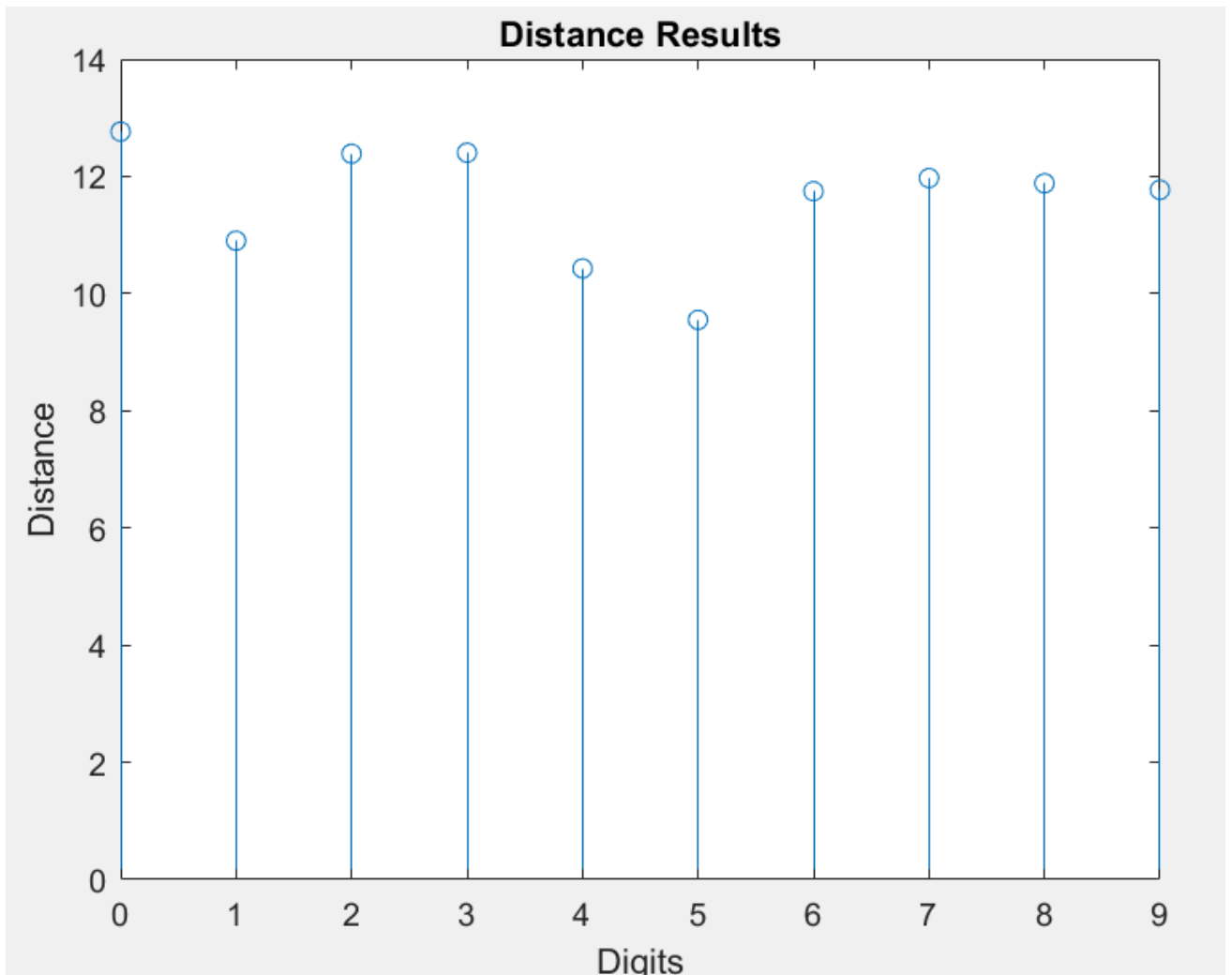


Fig.35: Distance results for digit 5

3.7 FAST FOURIER TRANSFORM

In this module's hardware implementation, it is required to use the FFT block built-in Vivado. By using this block it is expected to take the fast fourier transform of the inputs from the lab window. By taking the 512 point FFT and doing the overlapping of 256 points provided by the lab control, it is expected to have both real and imaginary values of the inputs at the result of the lab. Unfortunately, even though we understand what we need to do and know how to implement it. We could not implement this lab due to lack of lab ADC at that time. We mainly focused on the issues on the lab PCB and ADC hence, did not try to fix the absenteeism of the lab FFT.

3.8 MEL FILTER BANKS

As stated in the lab manual, 32 filter energies should be implemented in MATLAB and saved to a ROM of the Vivado. The frequency should range from 100 Hz to 4 KHz to ensure the spoken voice is captured. Each filter is shaped like triangles. The data shall be read through the lab FFT's output and multiplied using the multiplier block in the Vivado. Then the result of the multiplication should be stored in a RAM to further use this data in the lab DCT. As a result of failure in lab FFT, we did not implement this lab but were aware of the operations included. If we could have managed the time better and were 3 people, we believe we could successfully reach this stage of the project.

3.9 DISCRETE COSINE TRANSFORM

In this module, the energy data coming from lab mel's RAM is expected to be converted by the cosine transform in the lab dct. The result of this transform shall be stored in a dual-port RAM from the IP catalogue. In order to do that implementation, cosine coefficients should be generated using MATLAB or another software then these coefficients shall be applied to the results in lab mel frames via multiplication. This operation can be done with the multiplier block of the Vivado. In order to complete this lab fully, it is required to show different frequency values. However, as we have stated earlier, since we lost too much time on PCB and ADC we were not able to handle the time and could not implement this lab either.

3.10 COMPARE

As stated in the section 2 of this report, lab compare is the hardware implementation of the lab MATLAB. In this lab we have expected to see the results for spoken number recognition via hardware. For this lab multiple IP cores of RAM is needed to store the recorded spoken digits and one is needed for storing the testing sample. After acquiring the test sample, euclidean distance of the sample between the recorded digits should be calculated and the minimum distance shall give the result for the spoken digit. In this lab, ADC, window, control, PCB, fft, mel and dct must be working simultaneously. Unfortunately, we could not implement this lab but we were aware of its necessities and requirements. Also, we know what shall be done in order to accomplish this lab however, due to multiple problems encountered we did not implement it.

4 DISCUSSION AND CONCLUSION

4.1 Conclusion

We started the semester by first implementing the lab debug assignment. Being away from VHDL for almost 2 years caused us some troubles to implement this lab. We both studied some VHDL and inspected our past implementations in EEE102 class. We successfully implemented the lab on time then moved on with the lab control assignment. While implementing this lab, we have realized that our third group member is not going to do the project and give up on the project. Even though our third member left the lecture, we did not stop implementing the project and successfully completed the lab control assignment. Then in the 3rd lab with

the absence of our group and the heavy workload of the semester, we could not be able to successfully implement the lab ADC. Then with the accumulated weights of the oncoming labs, we have fallen behind the schedule. However, we were able to successfully accomplish the lab window, using the IP catalogue of Vivado instead of using the ADC. Then we successfully built a PCB that can satisfy the given requirements in the lab manual. We printed 2 boards because the first one failed to achieve the requirements of the lab. Also, biasing the electret microphone was a challenge for us since we have burnt multiple microphones. Then we successfully implement the lab MATLAB by using the short-time fourier transform. We were able to recognize 95% of the spoken digits correctly. However, after MATLAB we could not cope with the accumulated weights of the labs again since we did not have any lab ADC. We knew all the labs necessities and requirements; however, we were not able to implement the hardware design. Overall, we were not able to implement the hardware result of the project. We presented our MATLAB implementation with a great success rate and we can easily say that we were aware of every lab's intention.

4.2 Encountered Problems

One problem we have encountered and affected our performance radically is that our group member did not show up to the lectures and the lab sessions. This resulting that we have worked for the project as a 2 people group which set us back crucially. Due to our heavy load of the semester in order to graduate from the school on time, we could not cope with the burden of this project. This was not a 2 man job but we insisted on keep going after lab control but end up failing in the week of lab ADC. If we have known earlier, we could have contacted the teaching assistants and split into different groups.

Another issue we have encountered is that we have underestimated the workload of this course. We thought that we can handle this project on Saturdays however, that was not the case. We realize that we have forgotten and actually never learned VHDL correctly. Hence, it was very challenging for us to implement the design codes and testbench codes. Also with the workload of other courses, we could not be able to handle this process well. We lacked time management and process organization. This project taught us that it is really important to schedule the timings accurately.

One other issue was about the lab PCB. In this lab we have printed 2 different boards and circuits. Our first implementation was lack of amplifying the input voice signal between the desired voltage levels. Also, while biasing the microphones, we believe we burned down multiple microphones because the circuit was functioning well when the signal generator was used instead of the microphone. In our second print, we were able to overcome the biasing issue but the capacitors in the labs caused us some troubles. The capacitors was not in the same capacitance as they were labeled and we realized it very lately. Hence, lab PCB consumed a lot of time for us. In addition to that since we could not implement the lab ADC, the PCB was a meaningless effort at that time. We could have been focus on the simulation of other labs. Also, since we were only 2 people we cannot do the task division properly and we both tried to implement all the labs independently.

Also combining different sub-blocks were an issue that we did not expect. It was very challenging to combine 2 modules together let alone the whole system. We thought that we can simply just combine them in

a top module but it needed more work then we have imagined. Our lack of experience in VHDL, set us back in this process.

4.3 Future Work

A future work in the design could be simplifying the projects needs. It requires a lot of VHDL knowledge that we lack. In our implementation a future work of implementing the lab ADC would be beneficial. Adding the ADC module successfully to our PCB design and implementing the uncompleted labs of FFT, MEL, DCT and compare with the help of ADC module could lead us to success.

5 REFERENCES

- [1] E. Deruty, “Intuitive understanding of mfccs,” Medium, <https://medium.com/@deruty/sl/intuitive-understanding-of-mfccs-836d36a1f779> (accessed Jan. 10, 2025).
- [2] E. Pena and M. G. Legaspi, “UART: A Hardware Communication Protocol Understanding Universal Asynchronous Receiver/Transmitter | Analog Devices,” *Analog.com*, 2024. <https://www.analog.com/en/resources/analog-dialogue/articles/uart-a-hardware-communication-protocol.html>
- [3] “Altium Designer - PCB Design Software,” *www.altium.com*. <https://www.altium.com/altium-designer>
- [4] “ADC121S101,” *Ti.com*, 2019. <https://www.ti.com/product/ADC121S101> (accessed Jan. 10, 2025).
- [5] “AMD Vivado™ Design Suite,” *AMD*, Aug. 23, 2024. <https://www.amd.com/en/products/software/adaptive-socs-and-fpgas/vivado.html>
- [6] “Spectrogram and the STFT,” *speechprocessingbook*, 2025. https://speechprocessingbook.aalto.fi/Representations/Spectrogram_and_the_STFT.html (accessed Jan. 10, 2025).

6 APPENDICES

debug.vhd

```
library IEEE;
```

```
use IEEE.STD_LOGIC_1164.ALL;
```

```
entity top_module is
```

```
    Generic(N: integer:= 14);
```

```
Port (
```

```
    reset_in : in STD_LOGIC;
```

```
    clock_in : in STD_LOGIC;
```

```
    txd_out: out STD_LOGIC;
```

```
    start_in: in STD_LOGIC;
```

```
    ready_out: out STD_LOGIC;
```

```
);
```

```
end top_module;
```

```
architecture Behavioral of top_module is
```

```
    component rom is
```

```
    PORT (
```

```
        clka : IN STD_LOGIC;
```

```
        addra : IN STD_LOGIC_VECTOR(13 DOWNTO 0);
```

```
        douta : OUT STD_LOGIC_VECTOR(31 DOWNTO 0)
```

```
    );
```

end component;

component DEBUG is

Generic(N: integer:= 14);

Port (

reset_in: IN STD_LOGIC;

clock_in: IN STD_LOGIC;

mem_addr_out: OUT STD_LOGIC_VECTOR (N-1 downto 0);

mem_data_in: in STD_LOGIC_VECTOR (31 downto 0);

txd_out: OUT STD_LOGIC;

start_in: in STD_LOGIC;

ready_out: out STD_LOGIC);

end component;

signal addra: STD_LOGIC_VECTOR (N-1 downto 0);

signal douta: STD_LOGIC_VECTOR (31 downto 0);

begin

ROM0: rom port map (

clka => clock_in,

addra => addra,

douta => douta

);

ADC: adctest

port map(clock_in => clock_in,


```
reset_in => reset_in,  
spi_clk_out => spi_clk,  
chip_sel_out => chipsel,  
spi_data_in => spi_data_in,  
start_in => start_in,  
ready_out => ready_out,  
uart_out=> txd_out);
```

DEBUG_COMP: DEBUG generic map(N => N)

```
port map (  
reset_in => reset_in,  
clock_in => clock_in,  
mem_addr_out => addra,  
mem_data_in => douta,  
txd_out => txd_out,  
start_in => start_in,  
ready_out => ready_out);
```

end Behavioral;

control.vhd

```
library IEEE;  
use IEEE.STD_LOGIC_1164.ALL;  
use IEEE.STD_LOGIC_ARITH.ALL;  
use IEEE.STD_LOGIC_UNSIGNED.ALL;
```

```
use IEEE.NUMERIC_STD.ALL;
```

```
entity Lab_CTRL is
```

```
Port (
```

```
    reset_in      : in std_logic;
```

```
    clock_in       : in std_logic;
```

```
    start_adc_out   : out std_logic;
```

```
    ready_adc_in    : in std_logic;
```

```
    addr_out        : out std_logic_vector(13 downto 0);
```

```
    start_window_out : out std_logic;
```

```
    ready_window_in  : in std_logic;
```

```
    start_debug_out   : out std_logic;
```

```
    ready_debug_in    : in std_logic;
```

```
    start_in          : in std_logic;
```

```
    ready_out         : out std_logic;
```

```
    state_out         : out std_logic_vector(2 downto 0)
```

```
);
```

```
end Lab_CTRL;
```

```
architecture Behavioral of Lab_CTRL is
```

```
    constant frame : std_logic_vector(13 downto 0) := "000010000000000";
```

```
    constant half  : std_logic_vector(13 downto 0) := "000001000000000";
```

```
signal addr    : std_logic_vector(13 downto 0) := (others => '0');
```

```
signal count   : integer := 0;
```

```
type state_type is (
```

```
    initial, START_ADC, WAIT_ADC, START_WINDOW, WAIT_WINDOW, START_DEBUG,  
    WAIT_DEBUG
```

```
);
```

```
signal state    : state_type := initial;
```

```
begin
```

```
process(clock_in, reset_in)
```

```
begin
```

```
    if reset_in = '1' then
```

```
        state <= initial;
```

```
        start_adc_out <= '0';
```

```
        start_window_out <= '0';
```

```
        start_debug_out <= '0';
```

```
        addr <= (others => '0');
```

```
        count <= 0;
```

```
    elsif rising_edge(clock_in) then
```

```
        case state is
```

when initial =>

if start_in = '1' then

start_adc_out <= '1';

state <= START_ADC;

end if;

ready_out <= '0';

when START_ADC =>

start_adc_out <= '0';

if ready_adc_in = '1' then

start_window_out <= '1';

state <= START_WINDOW;

else

state <= WAIT_ADC;

end if;

when WAIT_ADC =>

if ready_adc_in = '1' then

start_window_out <= '1';

state <= START_WINDOW;

end if;

```
when START_WINDOW =>
    start_window_out <= '0';
    if ready_window_in = '1' then
        start_debug_out <= '1';
        state <= START_DEBUG;
    else

        state <= WAIT_WINDOW;
    end if;
```

```
when WAIT_WINDOW =>
    if ready_window_in = '1' then
        start_debug_out <= '1';
        state <= START_DEBUG;
    end if;
```

```
when START_DEBUG =>
    start_debug_out <= '0';
    if ready_debug_in = '1' then

        if count < 3 then
            addr <= addr + half;
```

```
        count <= count + 1;

        start_adc_out <= '1';

        state <= START_ADC;

    else

        ready_out <= '1';

        state <= initial;

    end if;

end if;
```

```
when others =>

    state <= initial;

end case;

end if;

end process;
```

```
addr_out <= addr;
```

```
state_out <= "000" when state = initial else

    "001" when state = START_ADC else

    "010" when state = WAIT_ADC else

    "011" when state = START_WINDOW else

    "100" when state = WAIT_WINDOW else
```

```
"101" when state = START_DEBUG else  
"110";
```

```
end Behavioral;
```

window.vhd

```
-----  
-- Company:  
-- Engineer:  
--  
-- Design Name:  
-- Module Name: LAB_WINDOW - Behavioral  
-- Project Name:  
-- Target Devices:  
-- Tool Versions:  
-- Description:  
--  
-- Dependencies:  
--  
-- Revision:  
-- Revision 0.01 - File Created  
-- Additional Comments:  
--
```

```
-----
```

```
library IEEE;

use IEEE.STD_LOGIC_1164.ALL;

use ieee.std_logic_unsigned.all;
```

entity LAB_WINDOW is

```
Port (

rst_in: in std_logic;

clk_in: in std_logic;

frm_addr_in: in std_logic_vector(13 downto 0);

adc_out_addr: out std_logic_vector(13 downto 0);

adc_in_data: in std_logic_vector(11 downto 0);

mem_in_addr: in std_logic_vector(8 downto 0);

mem_out_data: out std_logic_vector(19 downto 0);

strt_in: in std_logic;

rdy_out: out std_logic

);

end LAB_WINDOW;
```

architecture Behavioral of LAB_WINDOW is

component mult_gen_0 is

```
PORT (

CLK : IN STD_LOGIC;

A : IN STD_LOGIC_VECTOR(11 DOWNTO 0);

B : IN STD_LOGIC_VECTOR(7 DOWNTO 0);
```



```
    P : OUT STD_LOGIC_VECTOR(19 DOWNTO 0)
);
end component;
```

component blk_mem_gen_3 is

```
PORT (
    clka : IN STD_LOGIC;
    addra : IN STD_LOGIC_VECTOR(8 DOWNTO 0);
    douta : OUT STD_LOGIC_VECTOR(7 DOWNTO 0)
);
end component;
```

component blk_mem_gen_7 is

```
PORT (
    clka : IN STD_LOGIC;
    wea : IN STD_LOGIC_VECTOR(0 DOWNTO 0);
    addra : IN STD_LOGIC_VECTOR(8 DOWNTO 0);
    dina : IN STD_LOGIC_VECTOR(19 DOWNTO 0);
    clkb : IN STD_LOGIC;
    addrb : IN STD_LOGIC_VECTOR(8 DOWNTO 0);
    doutb : OUT STD_LOGIC_VECTOR(19 DOWNTO 0)
);
end component;
```

```
signal adc_data_sig: std_logic_vector(11 downto 0):=(others => '0');
```

```
signal coef_sig: std_logic_vector(7 downto 0):=(others => '0');
```

```
signal result_sig: std_logic_vector(19 downto 0):=(others => '0');
```

```
signal idx_sig: std_logic_vector(8 downto 0):=(others => '0');
```

```
signal hanning_out_data: std_logic_vector(7 downto 0):=(others => '0');
```

```
signal hanning_out_addr: std_logic_vector(8 downto 0):=(others => '0');
```

```
signal window_out_data: std_logic_vector(19 downto 0):=(others => '0');
```

```
signal window_out_addr: std_logic_vector(8 downto 0):=(others => '0');
```

```
signal clk_div: std_logic:='0';
```

```
signal strt_sig: std_logic:='0';
```

```
type state_enum is (IDLE, READ, MULT, WRITE, STOP);
```

```
signal current_state: state_enum := IDLE;
```

```
begin
```

```
multiply_inst: mult_gen_0 port map(
```

```
CLK => clk_in,
```

```
A => adc_data_sig,
```

```
B => coef_sig,
```

```
P => result_sig
```

```
);
```

```
hanning_mem: blk_mem_gen_3 port map(
```

```
clka => clk_in,
```

```
addra => hanning_out_addr,
douta => hanning_out_data
);
output_mem: blk_mem_gen_7 port map(
clka => clk_in,
wea => "1",
addra => window_out_addr,
dina => window_out_data,
clkb => clk_in,
addrb => mem_in_addr,
doutb => mem_out_data
);
```

```
process(clk_in)
begin
if rising_edge(clk_in) then
    clk_div <= not clk_div;

    if strt_in = '1' then
        strt_sig <= '1';
    end if;

    if strt_sig = '1' then
        strt_sig <= '0';
    end if;
end if;
```

```
end process;
```

```
process(clk_div)
```

```
begin
```

```
if rising_edge(clk_div) then
```

```
    if rst_in = '1' then
```

```
        current_state <= IDLE;
```

```
    else
```

```
        case current_state is
```

```
            when IDLE =>
```

```
                if strt_sig = '1' then
```

```
                    rdy_out <= '0';
```

```
                    current_state <= MULT;
```

```
                else
```

```
                    rdy_out <= '1';
```

```
                    idx_sig <= (others => '0');
```

```
                    adc_out_addr <= (others => '0');
```

```
                end if;
```

```
            when READ =>
```

```
                adc_out_addr <= idx_sig + frm_addr_in;
```

```
                hanning_out_addr <= idx_sig;
```

```
                current_state <= MULT;
```

when MULT =>

```
    adc_data_sig <= adc_in_data;  
    coef_sig <= hanning_out_data;  
    current_state <= WRITE;
```

when WRITE =>

```
    if idx_sig = "11111111" then  
        current_state <= STOP;  
    else  
        window_out_data <= result_sig;  
        window_out_addr <= idx_sig;  
        idx_sig <= idx_sig + 1;  
        current_state <= READ;  
    end if;
```

when STOP =>

```
    current_state <= IDLE;  
    rdy_out <= '1';
```

when others =>

```
    current_state <= IDLE;  
end case;
```

end if;

end if;

end process;

end Behavioral;

fft.vhd

mel.vhd

dct.vhd

compare.vhd

matlab.m

1. clc;
2. close all;
3. clear all;
4. %%
5. name = {'zero_audio_data.wav', 'one_audio_data.wav', 'two_audio_data.wav', 'three_audio_data.wav',
'four_audio_data.wav', 'five_audio_data.wav', 'six_audio_data.wav', 'seven_audio_data.wav',
'eight_audio_data.wav', 'nine_audio_data.wav'};
6. for i = 1:10
7. file = name{i};
- 8.
9. recObj = audiorecorder;

```
10. fprintf('_____Start speaking for %s _____ \n',file)
11. recordblocking(recObj, 2);
12. disp('End of recording.');
```

13.

```
14. thresh = 0.1;
15. voice_in = getaudiodata(recObj);
16. fs = recObj.SampleRate;
17.
18.
19.
```

```
20. voice_in(abs(voice_in) <= thresh) = 0;
21. firstNonZero = find(voice_in, 1);
22. voice_in = voice_in(firstNonZero:end);
23.
24. desiredLength = 16384;
25. voice_len = length(voice_in);
26. if voice_len < desiredLength
27.     padding = desiredLength - voice_len;
28.     voice_in = [voice_in; zeros(padding, 1)];
29. elseif voice_len > desiredLength
30.     voice_in = voice_in(1:desiredLength);
31. end
32.
33.
```

```
34. audiowrite(file, voice_in, fs);
35.
36. figure;
37. plot(voice_in)
38. title(file)
39. fprintf('%s saved\n', file)
40. end
41. %%
42. duration = 2;
43. mel_coeff = 32;
44. recObj = audiorecorder;
45. disp("Start speaking...");
46. recordblocking(recObj, duration);
47. disp("Recording completed!");
48. voice_in = getaudiodata(recObj);
49. fs = recObj.SampleRate;
50.
51. thresh = 0.1;
52. voice_in(abs(voice_in) <= thresh) = 0;
53. firstNonZero = find(voice_in, 1);
54. voice_in = voice_in(firstNonZero:end);
55. desiredLength = 16384;
56. voice_len = length(voice_in);
57. if voice_len < desiredLength
```



```

58. padding = desiredLength - voice_len;
59. voice_in = [voice_in; zeros(padding, 1)];
60. elseif voice_len > desiredLength
61. voice_in = voice_in(1:desiredLength);
62. end
63. voice_in = floor((voice_in .* (2^11)) + (2^11));
64. figure;
65. plot(voice_in);
66. title("New Record");
67. xlim([0,16384])
68. window = hann(512, "periodic");
69. window = floor((window .* (2^7)) + (2^7));
70. [S,F,T] = stft(voice_in, "Window", window, "OverlapLength", 256, "Centered", false);
71. S = floor((S .* (2^31)) + (2^31));
72. coeffs = mfcc(S, fs, "LogEnergy", "replace", "NumCoeffs", mel_coeff);
73. coeffs = floor((coeffs .* (2^15)) + (2^15));
74. logCoeffs = log(coeffs);
75. knownno = ["zero_audio_data.wav", "one_audio_data.wav", "two_audio_data.wav",
    "three_audio_data.wav", "four_audio_data.wav", "five_audio_data.wav", "six_audio_data.wav",
    "seven_audio_data.wav", "eight_audio_data.wav", "nine_audio_data.wav"];
76. numkwonno = numel(knownno);
77. knownCoeffs = cell(numkwonno, 1);
78. for i = 1:numkwonno
79. [knownAudio, knownFs] = audioread(knownno(i));
80. knownAudio = floor((knownAudio .* (2^11)) + (2^11));

```

```

81.
82. window = hann(512, "periodic");
83. window = floor((window .* (2^7)) + (2^7));
84. [S,F,T] = stft(knownAudio, "Window", window, "OverlapLength", 256, "Centered", false);
85. S = floor((S .* (2^31)) + (2^31));
86. coeffs = mfcc(S, knownFs, "LogEnergy", "replace", "NumCoeffs", mel_coeff);
87. coeffs = floor((coeffs .* (2^15)) + (2^15));
88. knownCoeffs{i} = log(coeffs);
89. end
90. distances = zeros(numkwonno, 1);
91. for i = 1:numkwonno
92.     knownCoeff = knownCoeffs{i};
93.     distances(i) = norm(dct(logCoeffs,10) - dct(knownCoeff,10),"fro");
94. end
95. [minDistance, identifiedNumber] = min(distances);
96. disp(distances)
97. fprintf("Min distance: %.2f\n", minDistance);
98. fprintf("The identified number is %d\n", identifiedNumber - 1);
99. %%
100. frameSize = round(0.03 * fs);
101. frame = voice_in(1:frameSize);
102. window = hann(frameSize);
103. windowedFrame = frame .* window;
104. figure;

```

```
105. subplot(2, 1, 1);
106. plot(frame);
107. title('Original Frame');
108. xlabel('Sample Index');
109. ylabel('Amplitude');
110. subplot(2, 1, 2);
111. plot(windowedFrame);
112. title('Windowed Frame');
113. xlabel('Sample Index');
114. ylabel('Amplitude');
115. %%
116. dctResults = dct(logCoeffs, 'Type', 2);
117. figure;
118. semilogy(dctResults);
119. title('DCT Results');
120. xlabel('Coefficient Index');
121. ylabel('Magnitude');
122. %%
123. energy = abs(S) .^ 2;
124. logEnergy = log(energy );
125. figure;
126. subplot(2, 1, 1);
127. plot( F,abs(S));
128. title('Energy Spectrum');
```

```

129. xlabel('Frequency (Hz)');
130. ylabel('Energy');
131. subplot(2, 1, 2);
132. plot(F,logEnergy);
133. title('Log Energy Spectrum');
134. xlabel('Frequency (Hz)');
135. ylabel('Log Energy');

```

Window.tb.vhd

```
library IEEE;
```

```
use IEEE.STD_LOGIC_1164.ALL;
```

```
use ieee.numeric_std.ALL;
```

```
entity TB_LAB_WINDOW is
```

```
end TB_LAB_WINDOW;
```

```
architecture Behavioral of TB_LAB_WINDOW is
```

```
    component LAB_WINDOW
```

```
    Port (
```

```
        reset_in: in std_logic;
```

```
        clock_in: in std_logic;
```

```
        frame_addr_in: in std_logic_vector(13 downto 0);
```

```
        adc_addr_out: out std_logic_vector(13 downto 0);
```

```
        adc_data_in: in std_logic_vector(11 downto 0);
```

```
        mem_addr_in: in std_logic_vector(8 downto 0);
```

```
    mem_data_out: out std_logic_vector(19 downto 0);  
    start_in: in std_logic;  
    ready_out: out std_logic  
);  
end component;
```

```
signal reset_in: std_logic := '0';  
signal clock_in: std_logic := '0';  
signal frame_addr_in: std_logic_vector(13 downto 0) := (others => '0');  
signal adc_addr_out: std_logic_vector(13 downto 0);  
signal adc_data_in: std_logic_vector(11 downto 0) := (others => '0');  
signal mem_addr_in: std_logic_vector(8 downto 0) := (others => '0');  
signal mem_data_out: std_logic_vector(19 downto 0);  
signal start_in: std_logic := '0';  
signal ready_out: std_logic;
```

```
constant clock_period: time := 10 ns;
```

```
begin
```

```
UUT: LAB_WINDOW
```

```
port map (
```

```
    reset_in => reset_in,
```

```
clock_in => clock_in,  
frame_addr_in => frame_addr_in,  
adc_addr_out => adc_addr_out,  
adc_data_in => adc_data_in,  
mem_addr_in => mem_addr_in,  
mem_data_out => mem_data_out,  
start_in => start_in,  
ready_out => ready_out  
);
```

```
clock_process: process  
begin  
    while true loop  
        clock_in <= '1';  
        wait for clock_period / 2;  
        clock_in <= '0';  
        wait for clock_period / 2;  
    end loop;  
end process;
```

```
stimulus: process  
begin
```

```
    reset_in <= '1';
```

```
wait for 2 * clock_period;
```

```
reset_in <= '0';
```

```
frame_addr_in <= "0000000000000001";
```

```
adc_data_in <= "0000000000000001";
```

```
start_in <= '1';
```

```
wait for clock_period;
```

```
start_in <= '0';
```

```
for i in 0 to 511 loop
```

```
    mem_addr_in <= std_logic_vector(to_unsigned(i, 9)); -- Increment memory address
```

```
    wait for clock_period;
```

```
end loop;
```

```
wait until ready_out = '1';
```

```
for i in 0 to 511 loop
```

```
    mem_addr_in <= std_logic_vector(to_unsigned(i, 9));
```

```
    wait for clock_period;
```

```
end loop;
```

```
wait for 100 * clock_period;
```

```
        assert false report "Simulation completed" severity failure;
    end process;
end Behavioral;
```

Debug.tb.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;
```

```
entity UART is
```

```
    Generic(
        N : integer := 14);
    Port(
        clock_in : in std_logic;
        reset_in : in std_logic;
        mem_addr_out : out std_logic_vector(N-1 downto 0);
        mem_data_in : in std_logic_vector(31 downto 0);
        txd_out : out std_logic;
        start_in : in std_logic;
        ready_out : out std_logic);
```

```
end UART;
```


architecture Behavioral of UART is

constant START_DATA : std_logic_vector(31 downto 0) := x"55AACC03";

constant END_DATA : std_logic_vector(31 downto 0) := x"AA5503CC";

type state_type is (steady, SEND_START, SEND_DATA, SEND_END);

signal state : state_type := steady;

signal address_counter : integer range 0 to (2**N)-1 := 0;

signal tx_data : std_logic_vector(31 downto 0);

signal tx_start : std_logic := '0';

signal tx_busy : std_logic := '0';

constant BAUD_DIV : std_logic_vector(15 downto 0) := x"0364";

component uart_tx

Port (

clk : in std_logic;

reset : in std_logic;

tx_start : in std_logic;

tx_data : in std_logic_vector(31 downto 0);

baud_rate : in std_logic_vector(15 downto 0);

tx : out std_logic;

tx_busy : out std_logic

);

end component;

```
begin
```

```
uart_tx_inst: uart_tx
```

```
port map (
```

```
    clk => clock_in,
```

```
    reset => reset_in,
```

```
    tx_start => tx_start,
```

```
    tx_data => tx_data,
```

```
    baud_rate => BAUD_DIV,
```

```
    tx => txd_out,
```

```
    tx_busy => tx_busy
```

```
);
```

```
process (clock_in, reset_in)
```

```
begin
```

```
    if reset_in = '1' then
```

```
        state <= steady;
```

```
        address_counter <= 0;
```

```
        tx_start <= '0';
```

```
        ready_out <= '1';
```

```
    elsif rising_edge(clock_in) then
```

```
        case state is
```

```
            when steady =>
```

```
if start_in = '1' then  
    tx_data <= START_DATA;  
    tx_start <= '1';  
    state <= SEND_START;  
    ready_out <= '0';  
end if;
```

```
when SEND_START =>  
    if tx_busy = '0' then  
        tx_start <= '0';  
        state <= SEND_DATA;  
        address_counter <= 0;  
    end if;
```

```
when SEND_DATA =>  
    if tx_busy = '0' then  
        if address_counter < (2**N) then  
            tx_data <= mem_data_in;  
            tx_start <= '1';  
            mem_addr_out <= std_logic_vector(to_unsigned(address_counter, N));  
            address_counter <= address_counter + 1;  
        else  
            tx_data <= END_DATA;  
            tx_start <= '1';  
            state <= SEND_END;
```

end if;

end if;

when SEND_END =>

if tx_busy = '0' then

tx_start <= '0';

state <= steady;

ready_out <= '1';

end if;

when others =>

state <= steady;

end case;

end if;

end process;

end Behavioral;

Control.tb.vhd

library IEEE;

use IEEE.STD_LOGIC_1164.ALL;

use IEEE.STD_LOGIC_ARITH.ALL;

use IEEE.STD_LOGIC_UNSIGNED.ALL;

use IEEE.NUMERIC_STD.ALL;

entity Lab_CTRL_tb is

end Lab_CTRL_tb;

architecture Behavioral of Lab_CTRL_tb is

```
signal reset_in      : std_logic := '0';
signal clock_in      : std_logic := '0';
signal start_adc_out  : std_logic;
signal ready_adc_in   : std_logic := '0';
signal frame_addr_out : std_logic_vector(13 downto 0);
signal start_window_out: std_logic;
signal ready_window_in : std_logic := '0';
signal start_fft_out  : std_logic;
signal ready_fft_in   : std_logic := '0';
signal start_mel_out  : std_logic;
signal ready_mel_in   : std_logic := '0';
signal start_dct_out  : std_logic;
signal ready_dct_in   : std_logic := '0';
signal start_comp_out : std_logic;
signal ready_comp_in  : std_logic := '0';
signal start_debug_out : std_logic;
signal ready_debug_in : std_logic := '0';
signal start_in       : std_logic := '0';
```

```
signal ready_out    : std_logic;
```

```
begin
```

```
DUT: entity work.Lab_CTRL
```

```
port map (
```

```
    reset_in      => reset_in,
```

```
    clock_in      => clock_in,
```

```
    start_adc_out  => start_adc_out,
```

```
    ready_adc_in   => ready_adc_in,
```

```
    frame_addr_out => frame_addr_out,
```

```
    start_window_out => start_window_out,
```

```
    ready_window_in => ready_window_in,
```

```
    start_fft_out  => start_fft_out,
```

```
    ready_fft_in   => ready_fft_in,
```

```
    start_mel_out  => start_mel_out,
```

```
    ready_mel_in   => ready_mel_in,
```

```
    start_dct_out  => start_dct_out,
```

```
    ready_dct_in   => ready_dct_in,
```

```
    start_comp_out => start_comp_out,
```

```
    ready_comp_in  => ready_comp_in,
```

```
    start_debug_out => start_debug_out,  
    ready_debug_in  => ready_debug_in,  
    start_in        => start_in,  
    ready_out       => ready_out  
);
```

```
clock_process : process
```

```
begin
```

```
    while true loop
```

```
        clock_in <= '0';
```

```
        wait for clock_period / 2;
```

```
        clock_in <= '1';
```

```
        wait for clock_period / 2;
```

```
    end loop;
```

```
end process;
```

```
begin
```

```
    reset_in <= '1';
```

```
    wait for 20 ns;
```

```
    reset_in <= '0';
```

```
    wait for 10 ns;
```

```
start_in <= '1';
```

```
wait for 10 ns;
```

```
start_in <= '0';
```

```
wait for 30 ns;
```

```
ready_adc_in <= '1';
```

```
wait for 10 ns;
```

```
ready_adc_in <= '0';
```

```
wait for 50 ns;
```

```
ready_window_in <= '1';
```

```
wait for 10 ns;
```

```
ready_window_in <= '0';
```

```
wait for 50 ns;
```

```
ready_fft_in <= '1';
```

```
wait for 10 ns;
```

```
ready_fft_in <= '0';
```

```
wait for 50 ns;
```



```
ready_mel_in <= '1';
```

```
wait for 10 ns;
```

```
ready_mel_in <= '0';
```

```
-
```

```
wait for 50 ns;
```

```
ready_dct_in <= '1';
```

```
wait for 10 ns;
```

```
ready_dct_in <= '0';
```

```
wait for 50 ns;
```

```
ready_comp_in <= '1';
```

```
wait for 10 ns;
```

```
ready_comp_in <= '0';
```

```
wait for 50 ns;
```

```
ready_debug_in <= '1';
```

```
wait for 10 ns;
```

```
ready_debug_in <= '0';
```

```
wait for 100 ns;
```

end process;

end Behavioral;

cons.xdc

This file is a general .xdc for the Basys3 rev B board

To use it in a project:

- uncomment the lines corresponding to used pins

- rename the used ports (in each line, after get_ports) according to the top level signal names in the project

Clock signal

set_property PACKAGE_PIN W5 [get_ports clk]

set_property IOSTANDARD LVCMOS33 [get_ports clk]

create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5} [get_ports clk]

Switches

set_property PACKAGE_PIN V17 [get_ports {start_in}]

set_property IOSTANDARD LVCMOS33 [get_ports {start_in}]

set_property PACKAGE_PIN V16 [get_ports {addr_in[0]}]

set_property IOSTANDARD LVCMOS33 [get_ports {addr_in[0]}]

set_property PACKAGE_PIN W16 [get_ports {addr_in[1]}]

set_property IOSTANDARD LVCMOS33 [get_ports {addr_in[1]}]

set_property PACKAGE_PIN W17 [get_ports {addr_in[2]}]

set_property IOSTANDARD LVCMOS33 [get_ports {addr_in[2]}]

```
set_property PACKAGE_PIN W15 [get_ports {addr_in[3]}]

    set_property IOSTANDARD LVCMOS33 [get_ports {addr_in[3]}]

#set_property PACKAGE_PIN V15 [get_ports {sw[5]}]

    #set_property IOSTANDARD LVCMOS33 [get_ports {sw[5]}]

#set_property PACKAGE_PIN W14 [get_ports {sw[6]}]

    #set_property IOSTANDARD LVCMOS33 [get_ports {sw[6]}]

#set_property PACKAGE_PIN W13 [get_ports {sw[7]}]

    #set_property IOSTANDARD LVCMOS33 [get_ports {sw[7]}]

#set_property PACKAGE_PIN V2 [get_ports {sw[8]}]

    #set_property IOSTANDARD LVCMOS33 [get_ports {sw[8]}]

#set_property PACKAGE_PIN T3 [get_ports {sw[9]}]

    #set_property IOSTANDARD LVCMOS33 [get_ports {sw[9]}]

#set_property PACKAGE_PIN T2 [get_ports {sw[10]}]

    #set_property IOSTANDARD LVCMOS33 [get_ports {sw[10]}]

#set_property PACKAGE_PIN R3 [get_ports {sw[11]}]

    #set_property IOSTANDARD LVCMOS33 [get_ports {sw[11]}]

#set_property PACKAGE_PIN W2 [get_ports {sw[12]}]

    #set_property IOSTANDARD LVCMOS33 [get_ports {sw[12]}]

#set_property PACKAGE_PIN U1 [get_ports {sw[13]}]

    #set_property IOSTANDARD LVCMOS33 [get_ports {sw[13]}]

#set_property PACKAGE_PIN T1 [get_ports {sw[14]}]

    #set_property IOSTANDARD LVCMOS33 [get_ports {sw[14]}]

#set_property PACKAGE_PIN R2 [get_ports {sw[15]}]

    #set_property IOSTANDARD LVCMOS33 [get_ports {sw[15]}]
```

LEDs

set_property PACKAGE_PIN U16 [get_ports {ready_ctrl}]

set_property IOSTANDARD LVCMOS33 [get_ports {ready_ctrl}]

set_property PACKAGE_PIN E19 [get_ports {ready_adc_out}]

set_property IOSTANDARD LVCMOS33 [get_ports {ready_adc_out}]

set_property PACKAGE_PIN U19 [get_ports {ready_win_out}]

set_property IOSTANDARD LVCMOS33 [get_ports {ready_win_out}]

set_property PACKAGE_PIN V19 [get_ports {ready_fft_out}]

set_property IOSTANDARD LVCMOS33 [get_ports {ready_fft_out}]

set_property PACKAGE_PIN W18 [get_ports {ready_mel_out}]

set_property IOSTANDARD LVCMOS33 [get_ports {ready_mel_out}]

set_property PACKAGE_PIN U15 [get_ports {ready_dct_out}]

set_property IOSTANDARD LVCMOS33 [get_ports {ready_dct_out}]

set_property PACKAGE_PIN U14 [get_ports {ready_cmp_out}]

set_property IOSTANDARD LVCMOS33 [get_ports {ready_cmp_out}]

#set_property PACKAGE_PIN V14 [get_ports {led[7]}]

#set_property IOSTANDARD LVCMOS33 [get_ports {led[7]}]

#set_property PACKAGE_PIN V13 [get_ports {led[8]}]

#set_property IOSTANDARD LVCMOS33 [get_ports {led[8]}]

#set_property PACKAGE_PIN V3 [get_ports {led[9]}]

#set_property IOSTANDARD LVCMOS33 [get_ports {led[9]}]

#set_property PACKAGE_PIN W3 [get_ports {led[10]}]

#set_property IOSTANDARD LVCMOS33 [get_ports {led[10]}]

#set_property PACKAGE_PIN U3 [get_ports {led[11]}]

```
#set_property IOSTANDARD LVCMOS33 [get_ports {led[11]]}

#set_property PACKAGE_PIN P3 [get_ports {led[12]]}

#set_property IOSTANDARD LVCMOS33 [get_ports {led[12]]}

#set_property PACKAGE_PIN N3 [get_ports {led[13]]}

#set_property IOSTANDARD LVCMOS33 [get_ports {led[13]]}

#set_property PACKAGE_PIN P1 [get_ports {led[14]]}

#set_property IOSTANDARD LVCMOS33 [get_ports {led[14]]}

#set_property PACKAGE_PIN L1 [get_ports {led[15]]}

#set_property IOSTANDARD LVCMOS33 [get_ports {led[15]]}
```

##7 segment display

```
set_property PACKAGE_PIN W7 [get_ports {led_out[6]]}

#set_property IOSTANDARD LVCMOS33 [get_ports {led_out[6]]}

set_property PACKAGE_PIN W6 [get_ports {led_out[5]]}

#set_property IOSTANDARD LVCMOS33 [get_ports {led_out[5]]}

set_property PACKAGE_PIN U8 [get_ports {led_out[4]]}

#set_property IOSTANDARD LVCMOS33 [get_ports {led_out[4]]}

set_property PACKAGE_PIN V8 [get_ports {led_out[3]]}

#set_property IOSTANDARD LVCMOS33 [get_ports {led_out[3]]}

set_property PACKAGE_PIN U5 [get_ports {led_out[2]]}

#set_property IOSTANDARD LVCMOS33 [get_ports {led_out[2]]}

set_property PACKAGE_PIN V5 [get_ports {led_out[1]]}

#set_property IOSTANDARD LVCMOS33 [get_ports {led_out[1]]}

set_property PACKAGE_PIN U7 [get_ports {led_out[0]]}
```

```
set_property IOSTANDARD LVCMOS33 [get_ports {led_out[0]}]
```

```
#set_property PACKAGE_PIN V7 [get_ports dp]
```

```
#set_property IOSTANDARD LVCMOS33 [get_ports dp]
```

```
set_property PACKAGE_PIN U2 [get_ports {led_slc[0]}]
```

```
set_property IOSTANDARD LVCMOS33 [get_ports {led_slc[0]}]
```

```
set_property PACKAGE_PIN U4 [get_ports {led_slc[1]}]
```

```
set_property IOSTANDARD LVCMOS33 [get_ports {led_slc[1]}]
```

```
set_property PACKAGE_PIN V4 [get_ports {led_slc[2]}]
```

```
set_property IOSTANDARD LVCMOS33 [get_ports {led_slc[2]}]
```

```
set_property PACKAGE_PIN W4 [get_ports {led_slc[3]}]
```

```
set_property IOSTANDARD LVCMOS33 [get_ports {led_slc[3]}]
```

```
##Buttons
```

```
set_property PACKAGE_PIN U18 [get_ports rst]
```

```
set_property IOSTANDARD LVCMOS33 [get_ports rst]
```

```
#set_property PACKAGE_PIN T18 [get_ports btnU]
```

```
#set_property IOSTANDARD LVCMOS33 [get_ports btnU]
```

```
set_property PACKAGE_PIN W19 [get_ports start]
```

```
set_property IOSTANDARD LVCMOS33 [get_ports start]
```

```
#set_property PACKAGE_PIN T17 [get_ports start_adc]
```

```
#set_property IOSTANDARD LVCMOS33 [get_ports start_adc]
```

```
#set_property PACKAGE_PIN U17 [get_ports start_dbg]
```

```
#set_property IOSTANDARD LVCMOS33 [get_ports start_dbg]
```

```
##Pmod Header JA
```

```
##Sch name = JA1
```

```
set_property PACKAGE_PIN J1 [get_ports {cs_n}]
```

```
    set_property IOSTANDARD LVCMOS33 [get_ports {cs_n}]
```

```
##Sch name = JA2
```

```
set_property PACKAGE_PIN L2 [get_ports {spi_data_in}]
```

```
    set_property IOSTANDARD LVCMOS33 [get_ports {spi_data_in}]
```

```
##Sch name = JA3
```

```
set_property PACKAGE_PIN J2 [get_ports {spi_clk}]
```

```
    set_property IOSTANDARD LVCMOS33 [get_ports {spi_clk}]
```

```
##Sch name = JA4
```

```
#set_property PACKAGE_PIN G2 [get_ports {JA[3]}]
```

```
    #set_property IOSTANDARD LVCMOS33 [get_ports {JA[3]}]
```

```
##Sch name = JA7
```

```
#set_property PACKAGE_PIN H1 [get_ports {JA[4]}]
```

```
    #set_property IOSTANDARD LVCMOS33 [get_ports {JA[4]}]
```

```
##Sch name = JA8
```

```
#set_property PACKAGE_PIN K2 [get_ports {JA[5]}]
```

```
    #set_property IOSTANDARD LVCMOS33 [get_ports {JA[5]}]
```

```
##Sch name = JA9
```

```
#set_property PACKAGE_PIN H2 [get_ports {JA[6]}]
```

```
#set_property IOSTANDARD LVCMOS33 [get_ports {JA[6]}]

##Sch name = JA10

#set_property PACKAGE_PIN G3 [get_ports {JA[7]}]

#set_property IOSTANDARD LVCMOS33 [get_ports {JA[7]}]


##Pmod Header JB

##Sch name = JB1

#set_property PACKAGE_PIN A14 [get_ports {JB[0]}]

#set_property IOSTANDARD LVCMOS33 [get_ports {JB[0]}]

##Sch name = JB2

#set_property PACKAGE_PIN A16 [get_ports {JB[1]}]

#set_property IOSTANDARD LVCMOS33 [get_ports {JB[1]}]

##Sch name = JB3

#set_property PACKAGE_PIN B15 [get_ports {JB[2]}]

#set_property IOSTANDARD LVCMOS33 [get_ports {JB[2]}]

##Sch name = JB4

#set_property PACKAGE_PIN B16 [get_ports {JB[3]}]

#set_property IOSTANDARD LVCMOS33 [get_ports {JB[3]}]

##Sch name = JB7

#set_property PACKAGE_PIN A15 [get_ports {JB[4]}]

#set_property IOSTANDARD LVCMOS33 [get_ports {JB[4]}]

##Sch name = JB8

#set_property PACKAGE_PIN A17 [get_ports {JB[5]}]
```



```
#set_property IOSTANDARD LVCMOS33 [get_ports {JB[5]}]

##Sch name = JB9

#set_property PACKAGE_PIN C15 [get_ports {JB[6]}]

#set_property IOSTANDARD LVCMOS33 [get_ports {JB[6]}]

##Sch name = JB10

#set_property PACKAGE_PIN C16 [get_ports {JB[7]}]

#set_property IOSTANDARD LVCMOS33 [get_ports {JB[7]}]
```

```
##Pmod Header JC

##Sch name = JC1

#set_property PACKAGE_PIN K17 [get_ports {JC[0]}]

#set_property IOSTANDARD LVCMOS33 [get_ports {JC[0]}]

##Sch name = JC2

#set_property PACKAGE_PIN M18 [get_ports {JC[1]}]

#set_property IOSTANDARD LVCMOS33 [get_ports {JC[1]}]

##Sch name = JC3

#set_property PACKAGE_PIN N17 [get_ports {JC[2]}]

#set_property IOSTANDARD LVCMOS33 [get_ports {JC[2]}]

##Sch name = JC4

#set_property PACKAGE_PIN P18 [get_ports {JC[3]}]

#set_property IOSTANDARD LVCMOS33 [get_ports {JC[3]}]

##Sch name = JC7

#set_property PACKAGE_PIN L17 [get_ports {JC[4]}]
```

```
#set_property IOSTANDARD LVCMOS33 [get_ports {JC[4]}]

##Sch name = JC8

#set_property PACKAGE_PIN M19 [get_ports {JC[5]}]

#set_property IOSTANDARD LVCMOS33 [get_ports {JC[5]}]

##Sch name = JC9

#set_property PACKAGE_PIN P17 [get_ports {JC[6]}]

#set_property IOSTANDARD LVCMOS33 [get_ports {JC[6]}]

##Sch name = JC10

#set_property PACKAGE_PIN R18 [get_ports {JC[7]}]

#set_property IOSTANDARD LVCMOS33 [get_ports {JC[7]}]


##Pmod Header JXADC

##Sch name = XA1_P

#set_property PACKAGE_PIN J3 [get_ports {JXADC[0]}]

#set_property IOSTANDARD LVCMOS33 [get_ports {JXADC[0]}]

##Sch name = XA2_P

#set_property PACKAGE_PIN L3 [get_ports {JXADC[1]}]

#set_property IOSTANDARD LVCMOS33 [get_ports {JXADC[1]}]

##Sch name = XA3_P

#set_property PACKAGE_PIN M2 [get_ports {JXADC[2]}]

#set_property IOSTANDARD LVCMOS33 [get_ports {JXADC[2]}]

##Sch name = XA4_P

#set_property PACKAGE_PIN N2 [get_ports {JXADC[3]}]

#set_property IOSTANDARD LVCMOS33 [get_ports {JXADC[3]}]
```

```
##Sch name = XA1_N

#set_property PACKAGE_PIN K3 [get_ports {JXADC[4]}]

    #set_property IOSTANDARD LVCMOS33 [get_ports {JXADC[4]}]

##Sch name = XA2_N

#set_property PACKAGE_PIN M3 [get_ports {JXADC[5]}]

    #set_property IOSTANDARD LVCMOS33 [get_ports {JXADC[5]}]

##Sch name = XA3_N

#set_property PACKAGE_PIN M1 [get_ports {JXADC[6]}]

    #set_property IOSTANDARD LVCMOS33 [get_ports {JXADC[6]}]

##Sch name = XA4_N

#set_property PACKAGE_PIN N1 [get_ports {JXADC[7]}]

    #set_property IOSTANDARD LVCMOS33 [get_ports {JXADC[7]}]
```

```
##VGA Connector
```

```
#set_property PACKAGE_PIN G19 [get_ports {vgaRed[0]}]

    #set_property IOSTANDARD LVCMOS33 [get_ports {vgaRed[0]}]

#set_property PACKAGE_PIN H19 [get_ports {vgaRed[1]}]

    #set_property IOSTANDARD LVCMOS33 [get_ports {vgaRed[1]}]

#set_property PACKAGE_PIN J19 [get_ports {vgaRed[2]}]

    #set_property IOSTANDARD LVCMOS33 [get_ports {vgaRed[2]}]

#set_property PACKAGE_PIN N19 [get_ports {vgaRed[3]}]

    #set_property IOSTANDARD LVCMOS33 [get_ports {vgaRed[3]}]

#set_property PACKAGE_PIN N18 [get_ports {vgaBlue[0]}]
```

```
#set_property IOSTANDARD LVCMOS33 [get_ports {vgaBlue[0]}]
#set_property PACKAGE_PIN L18 [get_ports {vgaBlue[1]}]
#set_property IOSTANDARD LVCMOS33 [get_ports {vgaBlue[1]}]
#set_property PACKAGE_PIN K18 [get_ports {vgaBlue[2]}]
#set_property IOSTANDARD LVCMOS33 [get_ports {vgaBlue[2]}]
#set_property PACKAGE_PIN J18 [get_ports {vgaBlue[3]}]
#set_property IOSTANDARD LVCMOS33 [get_ports {vgaBlue[3]}]
#set_property PACKAGE_PIN J17 [get_ports {vgaGreen[0]}]
#set_property IOSTANDARD LVCMOS33 [get_ports {vgaGreen[0]}]
#set_property PACKAGE_PIN H17 [get_ports {vgaGreen[1]}]
#set_property IOSTANDARD LVCMOS33 [get_ports {vgaGreen[1]}]
#set_property PACKAGE_PIN G17 [get_ports {vgaGreen[2]}]
#set_property IOSTANDARD LVCMOS33 [get_ports {vgaGreen[2]}]
#set_property PACKAGE_PIN D17 [get_ports {vgaGreen[3]}]
#set_property IOSTANDARD LVCMOS33 [get_ports {vgaGreen[3]}]
#set_property PACKAGE_PIN P19 [get_ports Hsync]
#set_property IOSTANDARD LVCMOS33 [get_ports Hsync]
#set_property PACKAGE_PIN R19 [get_ports Vsync]
#set_property IOSTANDARD LVCMOS33 [get_ports Vsync]
```

##USB-RS232 Interface

```
#set_property PACKAGE_PIN B18 [get_ports RsRx]
#set_property IOSTANDARD LVCMOS33 [get_ports RsRx]
set_property PACKAGE_PIN A18 [get_ports uart_txd]
```

```
set_property IOSTANDARD LVCMOS33 [get_ports uart_txd]
```

```
##USB HID (PS/2)
```

```
#set_property PACKAGE_PIN C17 [get_ports PS2Clk]
```

```
    #set_property IOSTANDARD LVCMOS33 [get_ports PS2Clk]
```

```
    #set_property PULLUP true [get_ports PS2Clk]
```

```
#set_property PACKAGE_PIN B17 [get_ports PS2Data]
```

```
    #set_property IOSTANDARD LVCMOS33 [get_ports PS2Data]
```

```
    #set_property PULLUP true [get_ports PS2Data]
```

```
##Quad SPI Flash
```

```
##Note that CCLK_0 cannot be placed in 7 series devices. You can access it using the
```

```
##STARTUPE2 primitive.
```

```
#set_property PACKAGE_PIN D18 [get_ports {QspiDB[0]}]
```

```
    #set_property IOSTANDARD LVCMOS33 [get_ports {QspiDB[0]}]
```

```
#set_property PACKAGE_PIN D19 [get_ports {QspiDB[1]}]
```

```
    #set_property IOSTANDARD LVCMOS33 [get_ports {QspiDB[1]}]
```

```
#set_property PACKAGE_PIN G18 [get_ports {QspiDB[2]}]
```

```
    #set_property IOSTANDARD LVCMOS33 [get_ports {QspiDB[2]}]
```

```
#set_property PACKAGE_PIN F18 [get_ports {QspiDB[3]}]
```

```
    #set_property IOSTANDARD LVCMOS33 [get_ports {QspiDB[3]}]
```

```
#set_property PACKAGE_PIN K19 [get_ports QspiCSn]
```

```
    #set_property IOSTANDARD LVCMOS33 [get_ports QspiCSn]
```

```
## Configuration options, can be used for all designs  
set_property CONFIG_VOLTAGE 3.3 [current_design]  
set_property CFGBVS VCCO [current_design]
```